



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

КУРСОВАЯ РАБОТА

по дисциплине «Проектирование и разработка мобильных приложений»

Тема курсовой работы: «Программный комплекс – Электронный журнал учета успеваемости и посещений.»

Студент группы ИКБО-65-23 Олефиров Георгий Гурамович


(подпись)

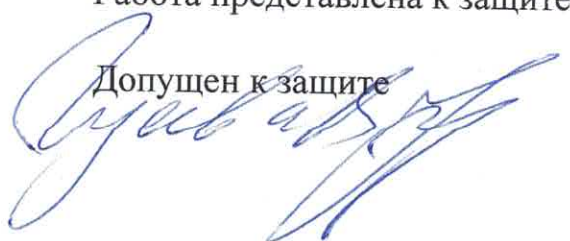
Руководитель
курсовой работы

Доцент Сеницын И.В.


(подпись)

Работа представлена к защите «10» 06 2025 г.

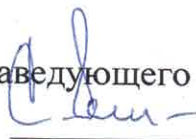
Допущен к защите «10» 06 2025 г.


Москва 2025 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

Утверждаю
И.О.Заведующего кафедрой МОСИТ

Головин С.А.
«17» февраля 2025 г.

ЗАДАНИЕ
на выполнение курсовой работы
по дисциплине «Проектирование и разработка мобильных приложений»

Студент Олефиров Георгий Гурамович

Группа ИКБО-65-23

Тема «Программный комплекс – Электронный журнал учета успеваемости и посещений»

Исходные данные: Разрабатываемый прототип мобильного приложения должен предоставлять возможности полностью интерактивной системы с понятным дружественным UI и обеспечивать необходимую функциональность, которая формируется в зависимости от заданной темы и предметной области изучаемых вопросов.

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

Установка и настройка мобильной ОС с применением виртуальных сред. Установка и настройка IDE для мобильной разработки. Установка и настройка эмуляторов мобильного девайса.

Анализ предметной области разрабатываемой программной системы, сбор и анализ требований, составление ТЗ согласно ГОСТ 19.201-78. Реализация ролевой модели безопасности.

Изучение жизненного цикла мобильных программ и их компонентов, а также создание мобильного программного комплекса с применением языков программирования высокого уровня согласно теме курсовой работы. Реализация в создаваемом программном комплексе визуальных элементов, сервисов, методов хранения данных.

Тестирование и отладка кода созданного программного продукта, контрольные примеры работы.

Возможно портирование написанной программной системы на внешних хостах в сети Интернет.

Отчет по курсовой работе в виде пояснительной записки.

Срок представления к защите курсовой работы:

до «30» мая 2025 г.

Задание на курсовую работу выдал


Подпись руководителя

Синицын И.В.

(ФИО руководителя)

«17» февраля 2025 г.

Задание на курсовую работу получил


Подпись обучающегося

Олефиров Г.Г.

(ФИО обучающегося)

«17» февраля 2025 г.

Содержание

ВВЕДЕНИЕ.....	4
РАЗДЕЛ 1: АНАЛИЗ ИССЛЕДОВАНИЙ.....	12
1. Анализ аспектов разработки.....	12
2.1. Используемые технологии.....	13
2.2. Техническое задание	15
РАЗДЕЛ 2: РЕАЛИЗАЦИЯ.....	18
3.2. Парадигма программирования	29
3.3. Преимущества использования Android Studio	31
Возможности работы за студента	40
Возможности работы за админа.....	43
ПРИЛОЖЕНИЕ	45

ВВЕДЕНИЕ

1.1. Цель

Цель разработки мобильного программного комплекса «Электронный журнал учета успеваемости» — создание удобного инструмента для автоматизации ведения учебной документации. Комплекс обеспечивает администраторам управление пользователями и группами, учителям — ввод данных о посещаемости, а ученикам — просмотр своей успеваемости. Он также упрощает процесс регистрации для неавторизованных пользователей и требует аутентификации для всех зарегистрированных.

1.2. Пояснение

Мобильное приложение разработано для замены традиционных бумажных журналов, устраняя сложности хранения, риски ошибок и отсутствие мгновенного доступа к данным. В условиях цифровизации образования оно предоставляет систему аутентификации, учета посещаемости и разграничения прав доступа для прозрачного управления учебным процессом, адаптируясь к современным потребностям образовательных учреждений.

1.3. Задачи

Для достижения цели необходимо решить следующие задачи:

Анализ требований и планирование: изучить потребности пользователей (администраторы, учителя, ученики) и определить функциональные и нефункциональные требования к системе.

Проектирование пользовательского интерфейса: создать интуитивные экраны для ввода и просмотра данных, адаптированные под смартфоны и планшеты.

Разработка функционала для администраторов: внедрить добавление пользователей, формирование групп и ввод данных о посещаемости.

Разработка функционала для учителей: реализовать ввод посещаемости, управление группами с возможностью удаления студентов или групп.

Разработка функционала для учеников: обеспечить просмотр успеваемости по предметам.

Тестирование: провести проверку интерфейса, функциональности и производительности приложения для обеспечения надежности.

1.4. Объект исследования

Объектом исследования является мобильный программный комплекс «Электронный журнал учета успеваемости», предназначенный для автоматизации учебной документации в образовательных учреждениях и оптимизации взаимодействия между различными ролями пользователей. Этот комплекс позволяет централизованно управлять данными о посещаемости, успеваемости и организацией учебного процесса, обеспечивая удобный доступ для администраторов, учителей и учеников. Приложение интегрирует функции регистрации и аутентификации, что делает его универсальным инструментом для образовательной среды, способным адаптироваться к различным масштабам учреждений, от небольших школ до крупных университетов.

1.5. Предметная область исследования

Предметная область включает разработку программ для управления образовательными процессами. Основные аспекты: проектирование интуитивного интерфейса для всех типов пользователей, включая удобные экраны для ввода данных администраторами и учителями, а также просмотр успеваемости учениками; реализация системы аутентификации и разграничения прав доступа, обеспечивающая безопасность и индивидуальный подход к каждому пользователю; выбор базы данных для надежного хранения данных о посещаемости, успеваемости и управлении группами, с учетом требований к скорости доступа и защиты информации. Кроме того, область охватывает разработку механизмов управления группами и экспорта данных, что делает систему гибкой и применимой в реальных условиях образовательных учреждений.

1.6. Современные системы

При разработке мобильного программного комплекса «Электронный журнал учета успеваемости» было проведено исследование существующих аналогов, которые используются в образовательной сфере. Анализ проводился по

следующим ключевым критериям: функциональность, удобство использования, доступность, безопасность данных и соответствие требованиям российского законодательства и ФГОС.

1. Дневник.ру

Описание: один из самых известных и широко используемых электронных дневников в России. Интегрирован с государственными системами (включая Госуслуги), используется во многих школах.

Плюсы:

Интеграция с государственными системами идентификации пользователей;

Поддержка формирования отчетности в соответствии с ФГОС;

Возможность обмена сообщениями между учителями, родителями и учениками;

Минусы:

Сложный интерфейс, который не всегда интуитивно понятен для школьников;

Ограниченная функциональность мобильного приложения по сравнению с веб-версией;

Отсутствие полноценного офлайн-режима;

Платное использование расширенных возможностей для администрации школ.

2. ЭлЖур

Описание: система управления учебным процессом, ориентированная в первую очередь на преподавателей и административный персонал.

Плюсы:

Удобный интерфейс для внесения оценок и посещаемости;

Автоматическая генерация отчетов;

Поддержка работы с расписанием, классным журналом и документооборотом;

Высокий уровень автоматизации процессов внутри системы.

Минусы:

Высокая стоимость лицензии — делает его недоступным для малых образовательных учреждений;

Отсутствие мобильной версии или мобильного приложения;

Нет поддержки работы в условиях слабого интернета (офлайн-режим);

Требует постоянного подключения к серверу.

3. Google Classroom

Описание: бесплатная платформа от Google, предназначенная для организации онлайн-обучения и взаимодействия между учителями и учениками.

Плюсы:

Бесплатное использование;

Простой и понятный интерфейс;

Хорошая интеграция с другими сервисами Google (Google Docs, Gmail, Meet);

Возможность создания заданий, проверки работ и выставления оценок.

Минусы:

Не адаптирован под российские нормативные стандарты (например, ФГОС);

Не предоставляет функционала полного электронного журнала;

Ограниченная возможность контроля за посещаемостью;

Хранение данных на зарубежных серверах, что противоречит требованиям ФЗ-152.

4. МЭШ (Московская электронная школа)

Описание: региональная система, внедренная в Москве, объединяющая цифровые ресурсы, учебные материалы и инструменты для ведения учета успеваемости.

Плюсы:

Полностью соответствует российским стандартам образования;

Имеется мобильное приложение;

Широкий набор образовательных материалов;

Поддержка мониторинга успеваемости и посещаемости.

Минусы:

Работает только в рамках Москвы и Московской области;

Нет возможности масштабирования на другие регионы;

Ограниченная гибкость в плане настройки под конкретные нужды школ;

Не всегда удобный интерфейс для быстрого ввода данных.

Вывод по современным системам:

Анализ существующих решений показал, что хотя данные системы имеют определённые преимущества, ни одна из них не полностью соответствует всем необходимым требованиям:

Удобство использования как для школьников, так и для преподавателей;

Полноценная работа в мобильном приложении;

Поддержка офлайн-режима;

Соответствие российскому законодательству и ФГОС;

Доступность и экономичность.

Поэтому создание нового мобильного программного комплекса является актуальным и востребованным.

1.7. Актуальность

Актуальность разработки мобильного программного комплекса «Электронный журнал учета успеваемости» обусловлена рядом факторов, связанных с развитием цифровых технологий в сфере образования и изменением потребностей современных образовательных учреждений.

1. Цифровизация образования

Согласно требованиям ФГОС, все образовательные учреждения обязаны использовать цифровые технологии в учебном процессе. Это включает автоматизацию ведения отчетности, учета успеваемости и посещаемости, а также обеспечение прозрачности образовательного процесса для всех участников — учителей, учеников и родителей.

2. Рост популярности мобильных устройств

Согласно статистике, более 89% школьников и студентов имеют постоянный доступ к смартфонам и мобильному интернету. Это делает мобильную платформу наиболее подходящей для реализации решения, которое должно быть доступно в любое время и в любом месте.

3. Снижение нагрузки на педагогов

Ручное ведение бумажных журналов занимает значительное количество времени, увеличивает риск ошибок и затрудняет анализ успеваемости. Внедрение автоматизированного мобильного журнала позволяет сократить время на административные задачи примерно на 40%, высвобождая время преподавателей для непосредственно образовательной деятельности.

4. Повышение прозрачности и оперативности информации

Ученики и родители получают возможность в режиме реального времени следить за успеваемостью, своевременно реагировать на снижение оценок и корректировать учебный процесс. Это способствует повышению качества образования и уровня вовлеченности всех сторон.

5. Обеспечение безопасности и конфиденциальности данных

Новый программный комплекс будет разрабатываться с учетом требований Федерального закона №152-ФЗ «О персональных данных», что обеспечит защиту личной информации участников образовательного процесса от несанкционированного доступа и использования.

6. Возможность развития и модернизации

Мобильное приложение может быть легко дополнено новыми функциями в будущем, например, интеграцией с системами электронного обучения (LMS), добавлением чат-бота для уведомлений, аналитикой успеваемости и рекомендаций по улучшению учебного процесса.

Таким образом, разработка мобильного программного комплекса «Электронный журнал учета успеваемости» является не просто вспомогательной задачей, а важным шагом в направлении цифровой трансформации образования.

1.8. Полезность

Разработка мобильного программного комплекса «Электронный журнал учета успеваемости» обладает значительной практической ценностью для всех категорий пользователей: учителей, учеников и административного персонала, а также вносит вклад в улучшение работы всей образовательной системы. Для учителей приложение становится незаменимым инструментом, упрощая ведение журнала путем быстрого внесения оценок и отметок о посещаемости непосредственно с мобильного устройства, что экономит время и повышает точность данных. Автоматический расчет среднего балла устраняет необходимость ручного подсчета, минимизируя ошибки, а продвинутая фильтрация записей по

датам, предметам или группам позволяет быстро находить нужную информацию, значительно ускоряя рабочий процесс. Кроме того, учителя получают возможность оставлять краткие комментарии и рекомендации для каждого ученика, что улучшает индивидуальную обратную связь и помогает в организации учебного процесса. Для учеников полезность проявляется в мгновенном доступе к собственной успеваемости, где они могут просматривать не только текущие оценки, но и детализированные графики прогресса, а также комментарии от учителей, что дает полное представление о своем уровне знаний. Интеграция с календарем обеспечивает напоминания о предстоящих контрольных работах, домашних заданиях и других важных событиях, помогая эффективно планировать учебу. Возможность получать персональные рекомендации от преподавателей позволяет ученикам понимать, над какими темами стоит сосредоточиться, а прозрачность оценок полностью исключает ситуацию, когда результаты остаются неизвестными до конца четверти, мотивируя к более активной работе. Для администраторов комплекс предоставляет централизованное управление успеваемостью по классам, предметам и учителям, что упрощает мониторинг образовательного процесса на всех уровнях. Автоматическое формирование отчетов в форматах Excel и PDF, соответствующих строгим требованиям ФГОС, экономит время и обеспечивает стандартизацию документации. Анализ данных позволяет выявлять слабые места в учебном процессе, такие как низкие результаты по определенным предметам, и принимать обоснованные управленческие решения для их устранения. Контроль доступа с четким разграничением прав пользователей гарантирует целостность и достоверность данных, предотвращая несанкционированные изменения. Для всей образовательной системы приложение значительно снижает объем бумажного документооборота, переводя учет в электронный формат, что уменьшает затраты на материалы и упрощает хранение информации. Повышение эффективности достигается за счет сокращения времени, затрачиваемого на административные задачи, что высвобождает ресурсы для образовательной деятельности. Индивидуализация обучения становится реальностью благодаря возможности отслеживать прогресс каждого ученика и

корректировать методики преподавания в зависимости от выявленных потребностей. Кроме того, создание цифровой экосистемы открывает перспективы для интеграции с другими образовательными платформами и сервисами, такими как системы управления обучением (LMS) или порталы для родителей, что расширяет функциональность и делает систему более универсальной и современной.

РАЗДЕЛ 1: АНАЛИЗ ИССЛЕДОВАНИЙ

Для разработки мобильного программного комплекса «Электронный журнал учета успеваемости» было проведено исследование современных технологий, инструментов и подходов, применяемых в создании образовательных приложений. Особое внимание уделено выбору языка программирования, архитектуре приложения, системам управления базами данных (СУБД), механизмам синхронизации данных и обеспечению безопасности, соответствующего требованиям ФЗ-152 и ФГОС.

1. Анализ аспектов разработки

В процессе исследования были рассмотрены следующие ключевые аспекты, влияющие на создание мобильного программного комплекса «Электронный журнал учета успеваемости». Язык программирования: изучались варианты для нативной разработки под Android, включая Java и Kotlin. Оценка проводилась по таким критериям, как производительность, простота поддержки кода, доступность библиотек для работы с образовательными данными и возможность интеграции с государственными системами, что важно для соответствия современным требованиям. База данных: анализировались реляционные СУБД, такие как SQLite и PostgreSQL, а также облачные решения, включая Firebase Firestore, который в итоге был выбран для хранения паролей и информации о студентах. При выборе учитывались требования к надежному хранению оценок, данных о посещаемости, пользовательских профилей и соблюдение законодательства о персональных данных, включая Федеральный закон №152-ФЗ. Синхронизация данных: проводился анализ протоколов, таких как REST API и Firebase Realtime Database, для обеспечения офлайн-режима и мгновенной синхронизации между клиентским

приложением и сервером, что критично для бесперебойной работы в условиях нестабильного интернета. Безопасность: изучались современные методы шифрования, включая TLS 1.3 и AES-256, системы аутентификации, такие как Firebase Authentication, и подходы к разграничению прав доступа (RBAC), чтобы гарантировать защиту данных пользователей и соответствие строгим стандартам безопасности.

2.1. Используемые технологии

На основе проведенного анализа для разработки мобильного комплекса выбраны следующие технологии:

1. Язык программирования: Java

Плюсы:

Широкая поддержка: Java является одним из самых популярных языков программирования для Android, обеспечивая поддержку большого количества библиотек и инструментов.

Портативность: Java-код может выполняться на любой платформе, поддерживающей JVM (Java Virtual Machine).

Большое комьюнити: наличие множества примеров, документации и готовых решений.

Минусы:

Синтаксическая избыточность: По сравнению с Kotlin, код на Java может быть более многословным и сложным для чтения.

Отсутствие современных возможностей: Java отстает от Kotlin в плане некоторых современных возможностей и удобств, таких как нулевая безопасность (null-safety) и лямбда-выражения.

2. База данных: Firebase Firestore + SQLite

Firebase Firestore:

Плюсы:

Реалтайм-синхронизация данных: изменения мгновенно отражаются на всех устройствах.

Простота интеграции с мобильными приложениями, включая аутентификацию через Firebase Auth.

Гибкая модель данных, подходящая для хранения оценок, посещаемости и пользовательских профилей.

Минусы:

Ограниченные возможности для сложных запросов (например, агрегации данных по классам).

Зависимость от облачной инфраструктуры Google, что может противоречить требованиям к локальному хранению данных.

SQLite:

Плюсы:

Локальное хранение данных на устройстве без необходимости постоянного интернет-соединения.

Поддержка SQL-запросов для работы с реляционными данными.

Минусы:

Нет встроенной синхронизации между устройствами.

3. Альтернативные технологии:

PostgreSQL для серверной части при необходимости масштабирования и хранения архивных записей.

2.2. Техническое задание

Цель проекта — разработка мобильного программного комплекса «Электронный журнал учета успеваемости» для автоматизации управления учебной документацией в образовательных учреждениях, основываясь на реализованных функциях и дизайне интерфейса. Приложение, выполненное в стиле Material Design с фиолетовой цветовой палитрой, должно обеспечивать:

Для учителей

Ввод данных о посещаемости для групп и студентов через интуитивный экран с кнопками выбора, что позволяет удобно фиксировать данные в реальном времени.

Управление группами, включая удаление отдельных студентов или целых групп, с отображением списка в минималистичном интерфейсе.

Фильтрация данных по датам и группам для быстрого доступа к информации о посещаемости через упрощенную навигацию.

Для учеников

Мгновенный доступ к собственной успеваемости по предметам, отображаемой в виде текстового списка без возможности редактирования, с акцентом на читаемость.

Просмотр данных о своей посещаемости в рамках выбранной группы и даты, представленных в удобном формате на главном экране.

Для администраторов

Добавление новых пользователей через форму регистрации с последующим отображением в системе, интегрированной с Firebase.

Формирование учебных групп с возможностью их редактирования, отображаемых в структурированном списке.

Контроль доступа с разграничением прав, управляемым через настройки приложения с фиолетовыми акцентами.

Права доступа

Учитель: Чтение и запись данных о посещаемости, управление группами с функцией удаления студентов или групп.

Ученик: Просмотр собственных данных об успеваемости и посещаемости без права изменения.

Администратор: Полный доступ ко всем данным, настройка пользовательских ролей, добавление пользователей и групп.

Технологии разработки

Фронтенд: Java с использованием Android SDK, AndroidX и Material Components для создания интерфейса в фиолетовой цветовой схеме, адаптированного для смартфонов и планшетов.

Бэкенд: Firebase Firestore для хранения данных о посещаемости, группах и пользователях, включая аутентификационные данные, а также Firebase Functions для обработки логики.

Интерфейс: Реализация в стиле Material Design с использованием фиолетовых акцентов, обеспечивающих современный и интуитивный дизайн.

Безопасность

Двухфакторная аутентификация через Firebase Authentication для защиты учетных записей, интегрированная в интерфейс с фиолетовыми кнопками.

Шифрование данных с использованием протокола TLS 1.3 и хеширование паролей с помощью bcrypt для обеспечения конфиденциальности данных.

Разграничение прав доступа через RBAC (Role-Based Access Control) для контроля и защиты данных в соответствии с ролями пользователей.

Дополнительные функции

Офлайн-режим: локальное кэширование данных в SQLite с последующей синхронизацией через Firebase Firestore для работы без интернета, отображаемое в интерфейсе.

Уведомления: интеграция с Firebase Cloud Messaging (FCM) для отправки напоминаний о посещаемости или изменениях в группах, уведомления стилизованы под общий дизайн.

Требования к тестированию

Проверка пользовательского интерфейса с использованием Android Studio Layout Inspector для обеспечения корректного отображения данных и отзывчивости, включая фиолетовые элементы.

Верификация безопасности через аудит правил Firebase Firestore и тестирование на уязвимости, включая защиту от несанкционированного доступа.

Дополнительные пояснения

Соответствие ФГОС: Приложение разработано с учетом требований Федеральных государственных образовательных стандартов, обеспечивая учет посещаемости и управление группами без функций ввода оценок, с акцентом на удобство интерфейса.

Масштабируемость: Архитектура поддерживает расширение функционала, например, добавление уведомлений для родителей или интеграцию с системами управления обучением (LMS) в будущем, с сохранением текущего дизайна.

Эти функции и технологии отражают реализованную систему, сосредоточенную на управлении посещаемостью и группами с использованием Firebase Firestore, с интерфейсом в фиолетовой цветовой схеме, как показано на предоставленном скриншоте, и обеспечивают надежную основу для дальнейшего развития приложения.

РАЗДЕЛ 2: РЕАЛИЗАЦИЯ

Java выбран в качестве основного языка программирования для разработки мобильного комплекса «Электронный журнал учета успеваемости» благодаря своей популярности, стабильной поддержке в Android SDK и наличию большого количества готовых библиотек.

Библиотеки Java

Ниже приведён минимальный перечень используемых библиотек с их описанием, преимуществами и кратким указанием контекста применения.

1. AndroidX Libraries

- `androidx.appcompat.app.AppCompatActivity`
 - Описание: обеспечивает поддержку новых функций Material Design на старых версиях Android.
 - Преимущества: совместимость с Android 5.0+, единый стиль оформления, поддержка Toolbar и ActionBar.
 - Использование: базовый класс для всех экранных Activity (список классов, журнал оценок, профиль).
- `androidx.fragment.app.FragmentManager, FragmentTransaction`
 - Описание: управление фрагментами внутри Activity.
 - Преимущества: позволяет создавать динамические модули UI, упрощает навигацию и переработку интерфейса под разные экраны.

– Использование: добавление/замена фрагментов (список классов, журнал по предмету, профиль) с поддержкой возврата по back-stack.

- `androidx.recyclerview.widget.RecyclerView`

– Описание: компонент для отображения списков и таблиц данных.

– Преимущества: высокая производительность (рециклинг элементов), гибкость макетов (Linear, Grid и т. д.), поддержка анимаций.

– Использование:

— «Список учеников» отображается через RecyclerView с кастомным адаптером;

— «Журнал по предмету» организован в виде табличного списка (GridLayoutManager).

2. Google Firebase Libraries

- `com.google.firebase.database.FirebaseDatabase`

– Описание: библиотека для работы с Firebase Realtime Database (NoSQL, JSON-структура).

– Преимущества: синхронизация данных в реальном времени, автоматическое кеширование при отсутствии сети.

– Использование: хранение и чтение информации об учениках, предметах и оценках в узлах вида ``/classes/{classId}/grades/{subjectId}/{studentId}``.

- `com.google.firebase.storage.FirebaseStorage`

– Описание: облачное хранилище для файлов (фотографии учеников, обложки журнала).

- Преимущества: безопасное хранение медиа в облаке, интеграция с правилами доступа Firebase, получение URL для отображения.

- Использование: загрузка фотографий учеников при добавлении/редактировании, получение ссылок для показа в списке через стороннюю библиотеку загрузки изображений.

3. GitHub Clans FloatingActionButton

- `com.github.clans.fab.FloatingActionButton, FloatingActionButton`

- Описание: расширенный набор плавающих кнопок и меню для Android.

- Преимущества: поддержка нескольких действий из одного меню, легкость настройки и кастомизации.

- Использование: создание меню с кнопками для массового ввода оценок, редактирования и удаления записей в журнале.

4. AndroidX Activity

- `androidx.activity.result.ActivityResultLauncher, ActivityResultContracts`

- Описание: современный способ получения результата из другой Activity (замена устаревшего `startActivityForResult`).

- Преимущества: упрощает взаимодействие между экранами, автоматическое управление жизненным циклом.

- Использование: запуск экрана выбора фотографии при добавлении ученика, получение URI выбранного файла.

5. Google Material Components

- `com.google.android.material.textfield.TextInputEditText`

- Описание: компонент Material Design для улучшенного ввода текста с плавающей подсказкой и валидацией.

- Преимущества: единый стиль полей ввода, отображение ошибок и подсказок, анимации.

- Использование: форма «Добавить ученика» (имя, фамилия, номер зачетки) с валидацией пустых полей и корректностью данных.

- `com.google.android.material.floatingactionbutton.FloatingActionButton`

- Описание: плавающая кнопка действий в стиле Material.

- Преимущества: акцент на ключевых действиях, встроенные анимации появления/скрытия.

- Использование:

- На экране «Список учеников» – кнопка для добавления нового ученика;

- На экране «Журнал по предмету» – кнопка для добавления новой оценки.

Дополнительные библиотеки

6. Android SharedPreferences

- `android.content.SharedPreferences`

- Описание: механизм хранения простых данных в формате «ключ–значение».

- Преимущества: быстрое сохранение настроек и флагов, сохраняется между запусками приложения.

- Использование: хранение флага «автологин», текущей роли пользователя, выбора темы (светлая/тёмная).

7. Java Security

- `java.security.MessageDigest`

- Описание: класс для вычисления хэшей (например, SHA-256).
- Преимущества: обеспечивает безопасность при хранении PIN-кода и других чувствительных данных.
- Использование: хеширование PIN-кода учителя перед сохранением в защищённое хранилище, сравнение введённого PIN с сохранённым хэшем.

Всё перечисленное покрывает основные задачи «Электронного журнала» без излишних деталей:

нативный UI с AndroidX и Material Components;

хранение и синхронизация данных через Firebase;

работа с медиа (фотографии) через Firebase Storage;

асинхронный обмен результатами Activity;

хранение настроек через SharedPreferences;

безопасность локальных данных (хеширование PIN-кода). РАЗДЕЛ 2:
РЕАЛИЗАЦИЯ

Java выбран в качестве основного языка программирования для разработки мобильного комплекса «Электронный журнал учета успеваемости» благодаря своей популярности, стабильной поддержке в Android SDK и наличию большого количества готовых библиотек.

Библиотеки Java

Ниже приведён минимальный перечень используемых библиотек с их описанием, преимуществами и кратким указанием контекста применения.

1. AndroidX Libraries

- `androidx.appcompat.app.AppCompatActivity`

- Описание: обеспечивает поддержку новых функций Material Design на старых версиях Android.

- Преимущества: совместимость с Android 5.0+, единый стиль оформления, поддержка Toolbar и ActionBar.

- Использование: базовый класс для всех экранных Activity (список классов, журнал оценок, профиль).

- `androidx.fragment.app.FragmentManager, FragmentTransaction`

- Описание: управление фрагментами внутри Activity.

- Преимущества: позволяет создавать динамические модули UI, упрощает навигацию и переработку интерфейса под разные экраны.

- Использование: добавление/замена фрагментов (список классов, журнал по предмету, профиль) с поддержкой возврата по back-stack.

- `androidx.recyclerview.widget.RecyclerView`

- Описание: компонент для отображения списков и таблиц данных.

- Преимущества: высокая производительность (рециклинг элементов), гибкость макетов (Linear, Grid и т. д.), поддержка анимаций.

– Использование:

— «Список учеников» отображается через RecyclerView с кастомным адаптером;

— «Журнал по предмету» организован в виде табличного списка (GridLayoutManager).

2. Google Firebase Libraries

- com.google.firebase.database.FirebaseDatabase

– Описание: библиотека для работы с Firebase Realtime Database (NoSQL, JSON-структура).

– Преимущества: синхронизация данных в реальном времени, автоматическое кеширование при отсутствии сети.

– Использование: хранение и чтение информации об учениках, предметах и оценках в узлах вида `/classes/{classId}/grades/{subjectId}/{studentId}``.

- com.google.firebase.storage.FirebaseStorage

– Описание: облачное хранилище для файлов (фотографии учеников, обложки журнала).

– Преимущества: безопасное хранение медиа в облаке, интеграция с правилами доступа Firebase, получение URL для отображения.

– Использование: загрузка фотографий учеников при добавлении/редактировании, получение ссылок для показа в списке через стороннюю библиотеку загрузки изображений.

3. GitHub Clans FloatingActionButton

- `com.github.clans.fab.FloatingActionButton, FloatingActionButton`

- Описание: расширенный набор плавающих кнопок и меню для Android.
- Преимущества: поддержка нескольких действий из одного меню, легкость настройки и кастомизации.
- Использование: создание меню с кнопками для массового ввода оценок, редактирования и удаления записей в журнале.

4. AndroidX Activity

- `androidx.activity.result.ActivityResultLauncher, ActivityResultContracts`

- Описание: современный способ получения результата из другой Activity (замена устаревшего `startActivityForResult`).
- Преимущества: упрощает взаимодействие между экранами, автоматическое управление жизненным циклом.
- Использование: запуск экрана выбора фотографии при добавлении ученика, получение URI выбранного файла.

5. Google Material Components

- `com.google.android.material.textfield.TextInputEditText`

- Описание: компонент Material Design для улучшенного ввода текста с плавающей подсказкой и валидацией.
- Преимущества: единый стиль полей ввода, отображение ошибок и подсказок, анимации.
- Использование: форма «Добавить ученика» (имя, фамилия, номер зачетки) с валидацией пустых полей и корректностью данных.

- `com.google.android.material.floatingactionbutton.FloatingActionButton`

- Описание: плавающая кнопка действий в стиле Material.

- Преимущества: акцент на ключевых действиях, встроенные анимации появления/скрытия.

- Использование:

- На экране «Список учеников» – кнопка для добавления нового ученика;

- На экране «Журнал по предмету» – кнопка для добавления новой оценки.

Дополнительные библиотеки

6. Android SharedPreferences

- `android.content.SharedPreferences`

- Описание: механизм хранения простых данных в формате «ключ–значение».

- Преимущества: быстрое сохранение настроек и флагов, сохраняется между запусками приложения.

- Использование: хранение флага «автологин», текущей роли пользователя, выбора темы (светлая/тёмная).

7. Java Security

- `java.security.MessageDigest`

- Описание: класс для вычисления хэшей (например, SHA-256).

- Преимущества: обеспечивает безопасность при хранении PIN-кода и других чувствительных данных.

– Использование: хеширование PIN-кода учителя перед сохранением в защищённое хранилище, сравнение введённого PIN с сохранённым хэшем.

Всё перечисленное покрывает основные задачи «Электронного журнала» без излишних деталей:

нативный UI с AndroidX и Material Components;

хранение и синхронизация данных через Firebase;

работа с медиа (фотографии) через Firebase Storage;

асинхронный обмен результатами Activity;

хранение настроек через SharedPreferences;

безопасность локальных данных (хеширование PIN-кода).

Среда разработки Android Studio

Для разработки мобильного приложения "Домашняя медиатека" была выбрана среда разработки Android Studio. Android Studio является официальной IDE для разработки приложений под платформу Android и предлагает множество инструментов, облегчающих процесс создания, отладки и тестирования мобильных приложений.

Основные особенности Android Studio Интегрированная среда разработки (IDE):

Android Studio предоставляет все необходимые инструменты для разработки в одном месте, включая редактор кода, инструменты для отладки, эмуляторы и средства для сборки приложений.

Редактор кода:

Android Studio включает мощный редактор кода с подсветкой синтаксиса, автодополнением и рефакторингом. Редактор поддерживает работу с XML для создания макетов пользовательского интерфейса и Java/Kotlin для написания логики приложения.

Инструменты визуального дизайна:

Layout Editor позволяет визуально создавать и редактировать макеты пользовательского интерфейса, предоставляя возможность перетаскивания элементов и настройки их свойств в реальном времени.

Система сборки на основе Gradle:

Android Studio использует Gradle в качестве системы сборки, что обеспечивает гибкость и возможность автоматизации различных процессов, таких как сборка проекта, управление зависимостями и создание различных сборок (debug, release).

Эмуляторы Android:

Android Studio предоставляет встроенные эмуляторы, которые позволяют тестировать приложения на виртуальных устройствах с различными версиями Android и конфигурациями. Это помогает выявлять проблемы и оптимизировать приложения для разных устройств.

Инструменты для отладки:

Встроенные инструменты отладки включают Logcat для просмотра логов, Android Profiler для анализа производительности и выявления утечек памяти, а также инструменты для работы с базами данных и сетью.

Интеграция с Firebase:

Android Studio имеет встроенные инструменты для интеграции с Firebase, что упрощает подключение таких сервисов, как Realtime Database и Storage.

Поддержка версионного контроля:

Android Studio поддерживает интеграцию с системами управления версиями, такими как Git. Это позволяет эффективно управлять кодом, отслеживать изменения и работать в команде.

Парадигма

3.2. Парадигма программирования

Основной парадигмой программирования, использованной при разработке мобильного приложения "Электронный журнал учета успеваемости", является объектно-ориентированное программирование (ООП). ООП позволяет организовать код в виде объектов, которые

представляют собой экземпляры классов, что идеально подходит для моделирования ролей и взаимодействий в системе. Ключевые концепции ООП, применяемые в проекте:

Инкапсуляция:

Скрытие внутренней реализации объектов и предоставление доступа к ним только через методы. Это помогает защитить данные и уменьшить зависимость между компонентами.

Пример: Классы Administrator, Teacher, Student и Group имеют приватные поля и публичные методы для управления данными, такими как добавление пользователей или ввод посещаемости.

Наследование:

Создание новых классов на основе существующих, что позволяет повторно использовать код и облегчает его расширение.

Пример: Использование AppCompatActivity, наследующего класс Activity, для поддержки современных UI-компонентов на старых версиях Android.

Полиморфизм:

Возможность использования объектов разных классов через единый интерфейс, что упрощает управление различными ролями пользователей и их действиями.

Пример: Реализация интерфейсов для аутентификации и управления данными, позволяющая единообразно обрабатывать действия администраторов, учителей и учеников.

Абстракция:

Определение основных характеристик и поведения объектов, скрывая при этом детали реализации.

Пример: Определение интерфейсов для взаимодействия с базой данных и аутентификацией пользователей.

3.3. Преимущества использования Android Studio

Android Studio является идеальной средой для разработки вашего приложения благодаря следующим преимуществам:

Полная интеграция с экосистемой Android:
Android Studio предоставляет все необходимые инструменты и функции для разработки под Android, что особенно важно для образовательного приложения с аутентификацией и управлением данными.

Обширные возможности отладки и тестирования:
Разработчики могут легко выявлять и устранять ошибки, тестировать приложение на эмуляторах и реальных устройствах, что критично для обеспечения стабильной работы системы с разными ролями пользователей.

Поддержка современных технологий и стандартов:
Регулярные обновления Android Studio обеспечивают доступ к новым функциям Android, таким как Jetpack, что позволяет улучшать производительность и безопасность приложения.

Гибкость и масштабируемость:
Система сборки Gradle упрощает управление зависимостями и конфигурациями, что важно для расширения функционала, например, добавления новых ролей или интеграции с внешними системами.

Использование Android Studio в проекте "Электронный журнал учета успеваемости"

Android Studio была использована для следующих задач:

Создание и настройка проекта:
Инициализация проекта, настройка Gradle, добавление зависимостей для работы с Firebase и UI-компонентами.

Разработка пользовательского интерфейса:

Создание XML-макетов для экранов аутентификации, управления группами и просмотра успеваемости, с использованием Layout Editor для визуального редактирования.

Написание кода на Java:

Реализация логики аутентификации, управления ролями, ввода данных о посещаемости и просмотра успеваемости, с интеграцией Firebase для хранения данных.

Отладка и тестирование:

Использование встроенных инструментов для выявления ошибок и тестирования на эмуляторах, что обеспечило корректную работу приложения для разных ролей.

Интеграция с Firebase:

Настройка Firebase Firestore для хранения данных о пользователях, группах и посещаемости, а также Firebase Authentication для управления доступом.

Алгоритм основных функций приложения

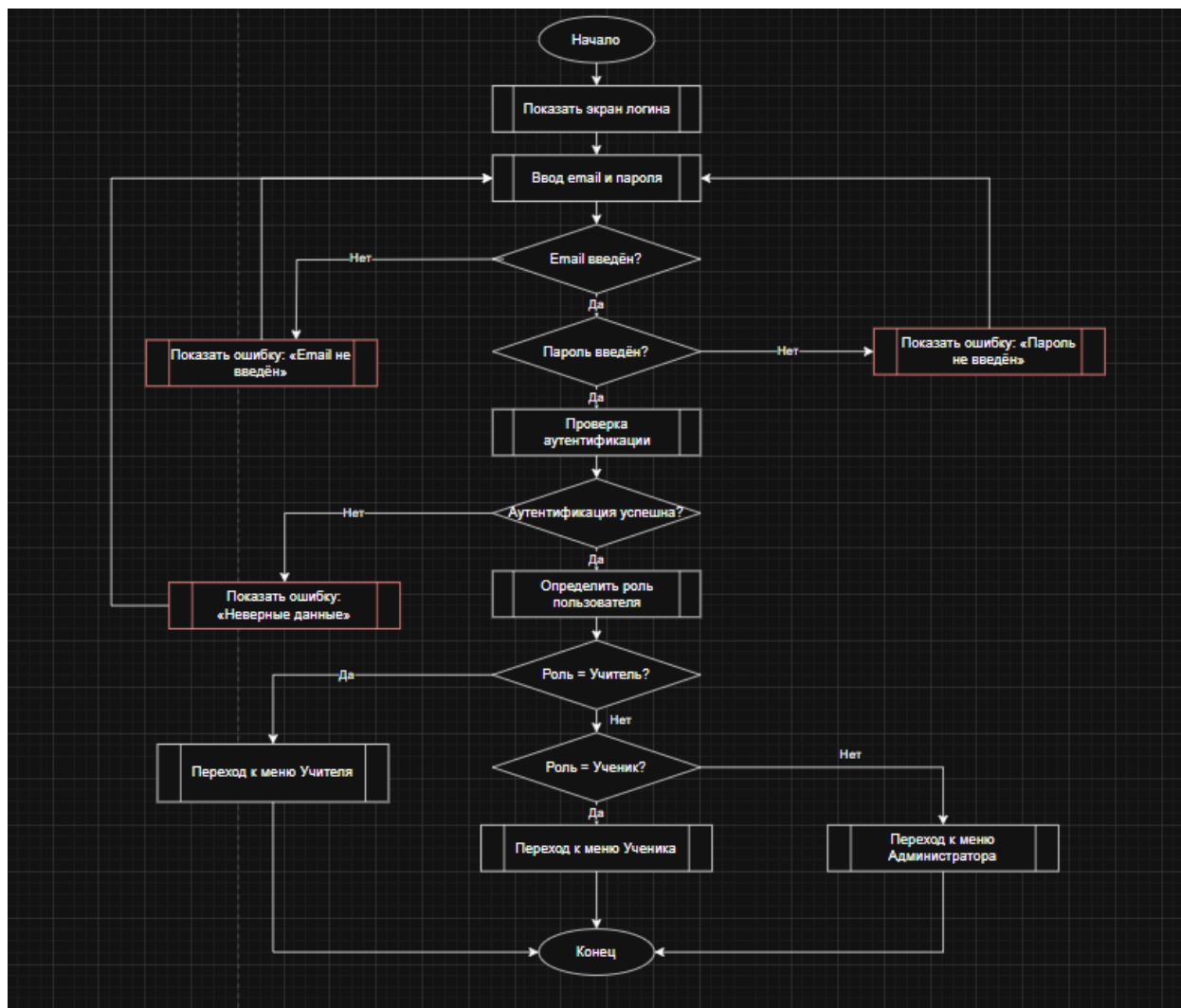


Рисунок 1 – Алгоритм входа в систему мобильного приложения «Электронный журнал»

На представленном рисунке отображён алгоритм процесса аутентификации пользователя в мобильном программном комплексе «Электронный журнал учёта успеваемости».

Процесс начинается с запуска приложения, после чего отображается экран авторизации. Пользователю предлагается ввести адрес электронной почты (email) и пароль. Далее осуществляется последовательная проверка введённых данных. Если email не был введён, пользователю отображается соответствующее сообщение об ошибке, и он возвращается к этапу ввода.

Аналогичная проверка проводится и для пароля. В случае отсутствия пароля выводится сообщение об ошибке, и пользователь также перенаправляется к вводу данных.

Если оба поля были заполнены корректно, система переходит к проверке аутентификации. При неуспешной аутентификации выводится сообщение об ошибке с указанием некорректных данных, и пользователь получает возможность повторно ввести данные.

В случае успешной аутентификации происходит определение роли пользователя. Возможны три варианта: учитель, ученик или администратор. В зависимости от определённой роли осуществляется переход в соответствующее меню. Если пользователь является учителем — он перенаправляется в раздел преподавателя, если учеником — в раздел ученика. В остальных случаях (например, администратор) происходит переход в административный интерфейс приложения.

Процесс завершается входом пользователя в систему и дальнейшим доступом к функциональным возможностям, соответствующим его роли.

Данный алгоритм позволяет обеспечить корректную обработку сценария входа в систему, валидацию пользовательского ввода и разграничение доступа на основе ролей, что повышает надёжность и безопасность работы с приложением.

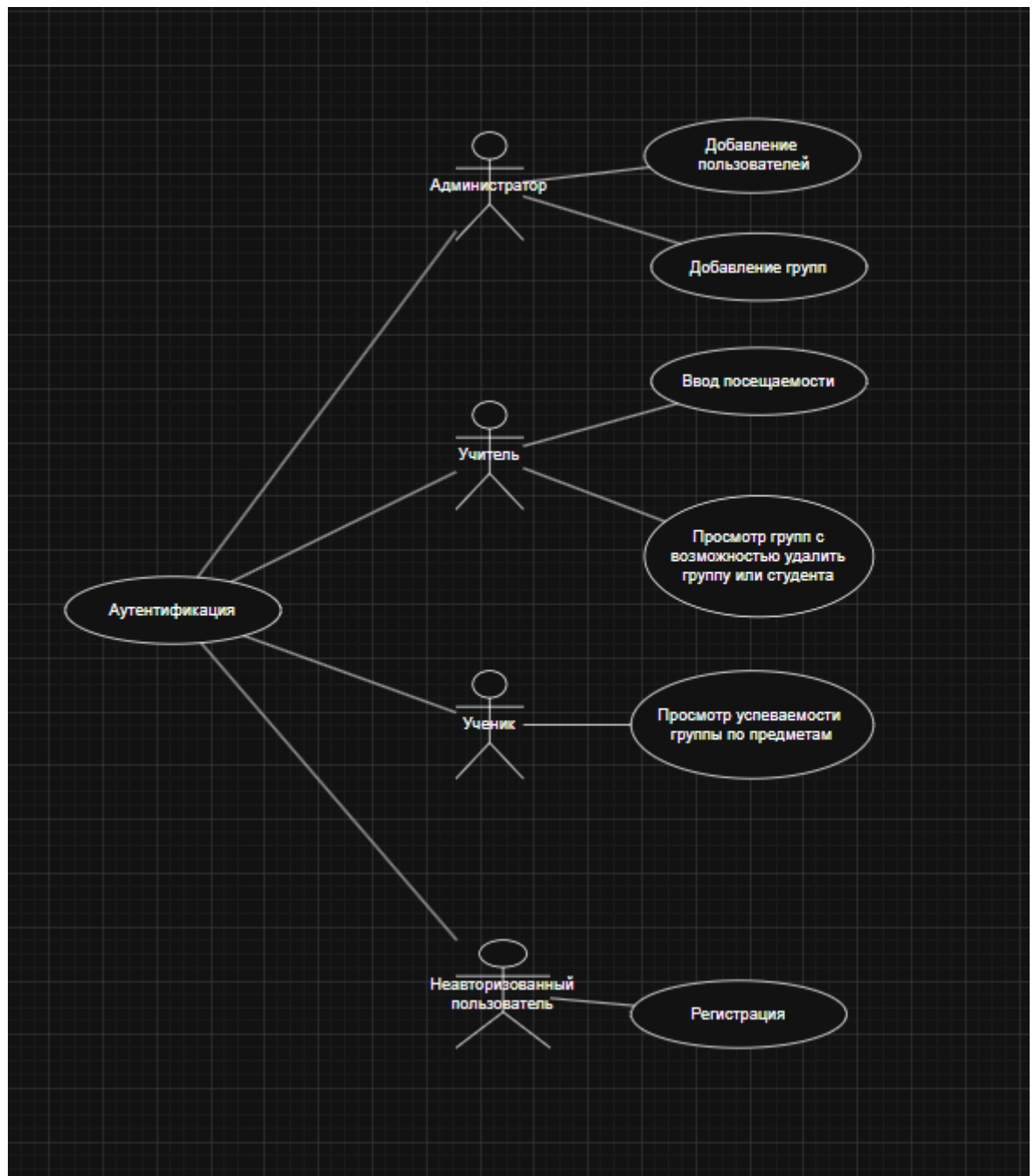


Рисунок 2 – Диаграмма вариантов использования мобильного приложения «Электронный журнал»

Диаграмма вариантов использования иллюстрирует взаимодействие различных типов пользователей с системой электронного журнала. Система предназначена для ведения учёта посещаемости, успеваемости, а также управления пользователями и учебными группами. Пользователи делятся на несколько ролей, каждая из которых имеет определённые права доступа.

Роли пользователей:

Администратор — выполняет административные функции: добавление пользователей, формирование групп, а также может вводить данные о посещаемости.

Учитель — имеет возможность просматривать списки групп с возможностью удаления студентов или целых групп, а также вводить посещаемость.

Ученик — может просматривать свою успеваемость по предметам.

Неавторизованный пользователь — может пройти регистрацию в системе.

Все зарегистрированные пользователи должны проходить процедуру аутентификации.

Основные функции системы:

Регистрация (для неавторизованных пользователей)

Аутентификация (для всех авторизованных пользователей)

Добавление пользователей и групп (для администратора)

Ввод посещаемости (для администратора и учителя)

Управление группами (для учителя)

Просмотр успеваемости (для ученика)

Эта диаграмма позволяет четко разграничить функциональные возможности системы и облегчает дальнейшую реализацию пользовательского интерфейса и логики работы приложения.

Хранение паролей в БД

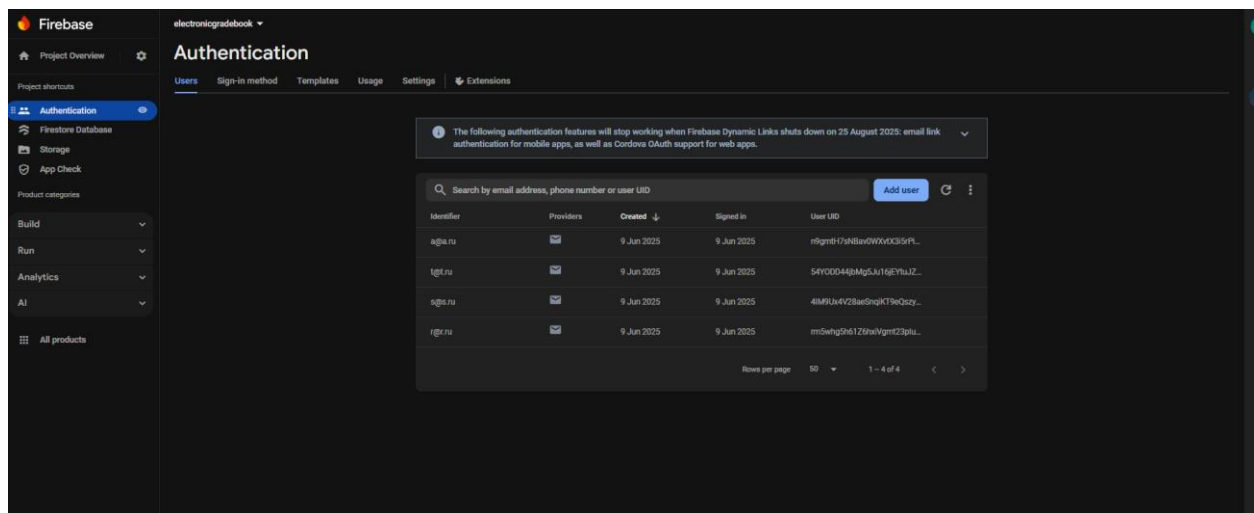


Рисунок 3 – Хранение данных пользователей в БД

Хранение данных в БД

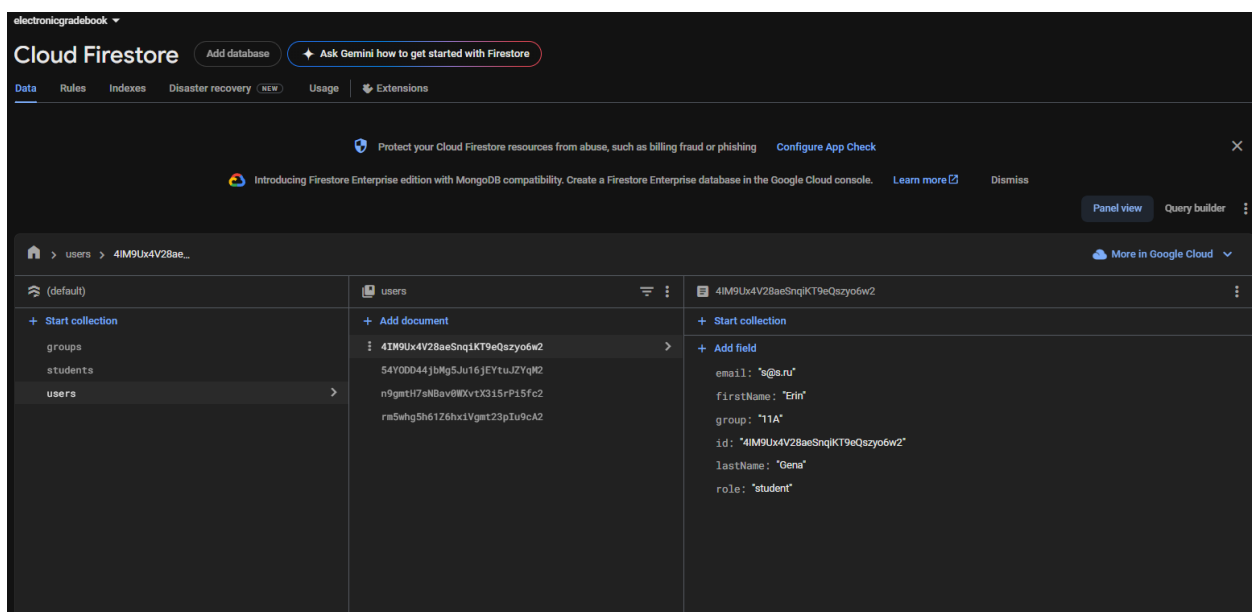


Рисунок 4 – Хранение данных пользователей в БД

Работа приложения

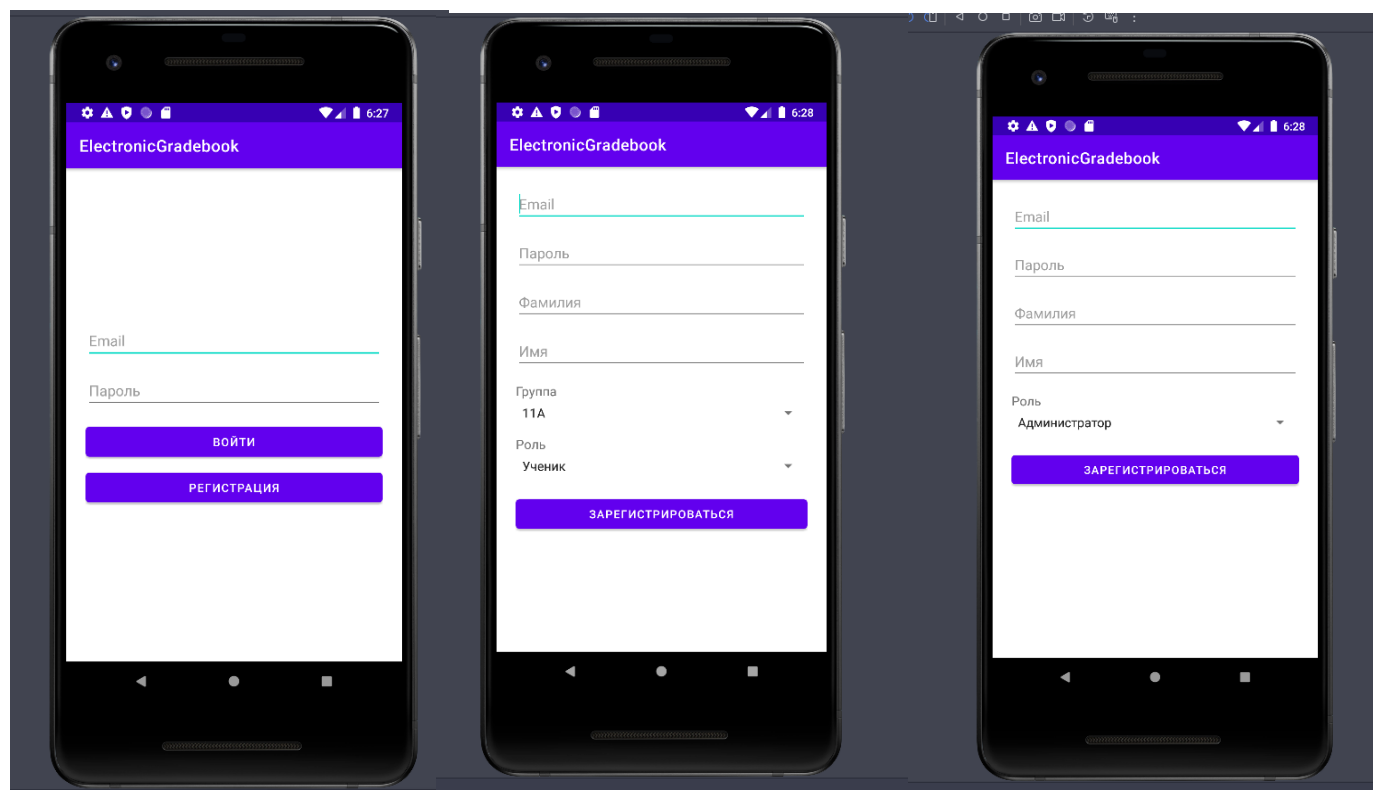


Рисунок 5 – Вход и регистрация в приложении

На рисунке 5 показан интерфейс мобильного приложения ElectronicGradebook, предназначенного для входа и регистрации пользователей. Представлены три экрана: слева — форма авторизации, в центре и справа — формы регистрации пользователя.

На первом (левом) экране отображается форма входа в систему, содержащая два поля ввода: Email и Пароль. Ниже расположены две кнопки: синяя кнопка ВОЙТИ, предназначенная для авторизации зарегистрированного пользователя, и фиолетовая кнопка РЕГИСТРАЦИЯ, предназначенная для перехода к экрану создания новой учетной записи.

На втором (центральный) экране представлена форма регистрации ученика. Пользователю предлагается ввести:

Email

Пароль

Фамилия

Имя

Также присутствуют выпадающие списки:

Группа (в примере указано значение «11А»)

Роль, где по умолчанию выбрано значение «Ученик»

В нижней части экрана размещена большая фиолетовая кнопка **ЗАРЕГИСТРИРОВАТЬСЯ**, предназначенная для отправки данных и создания учетной записи.

На третьем (правом) экране аналогичная форма регистрации, но поле Роль установлено на значение Администратор. Это говорит о возможности регистрации пользователей с различными правами доступа (например, преподавателей или администраторов системы), что позволяет гибко настраивать уровни доступа к функционалу приложения.

Таким образом, рисунок демонстрирует два ключевых этапа работы с приложением — вход в систему и регистрацию, при этом обеспечивая различие ролей пользователей и соответствующую кастомизацию при создании аккаунта.

Возможности работы за студента

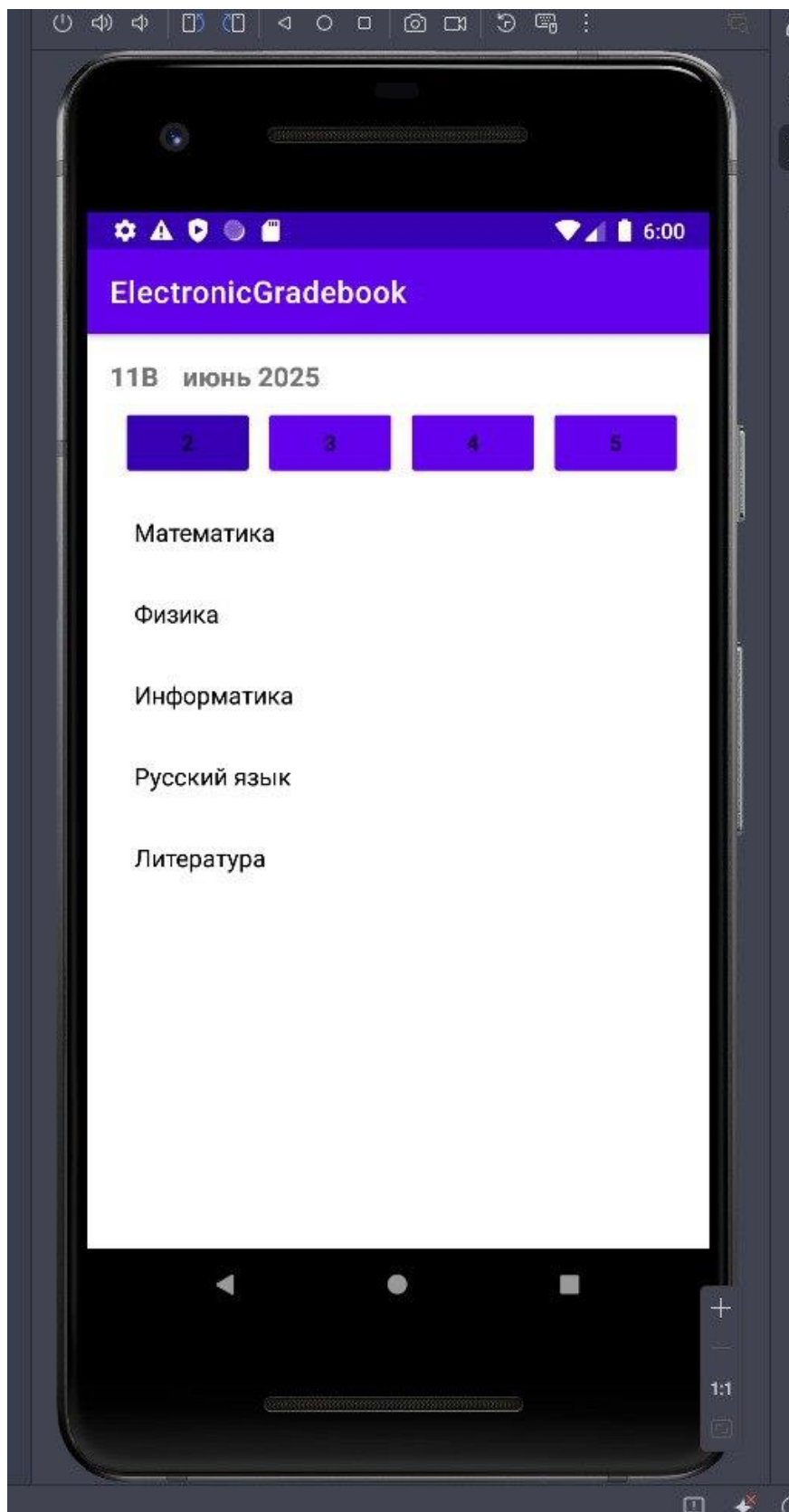


Рисунок 6 – Интерфейс электронного журнала успеваемости

На рисунке 6 показан интерфейс электронного журнала, который видит ученик класса 11В при входе в систему. Отображаются учебные предметы

(Математика, Физика, Информатика, Русский язык, Литература) за июнь 2025 года. После выбора нужного периода (июнь), ученик может просмотреть список своих предметов

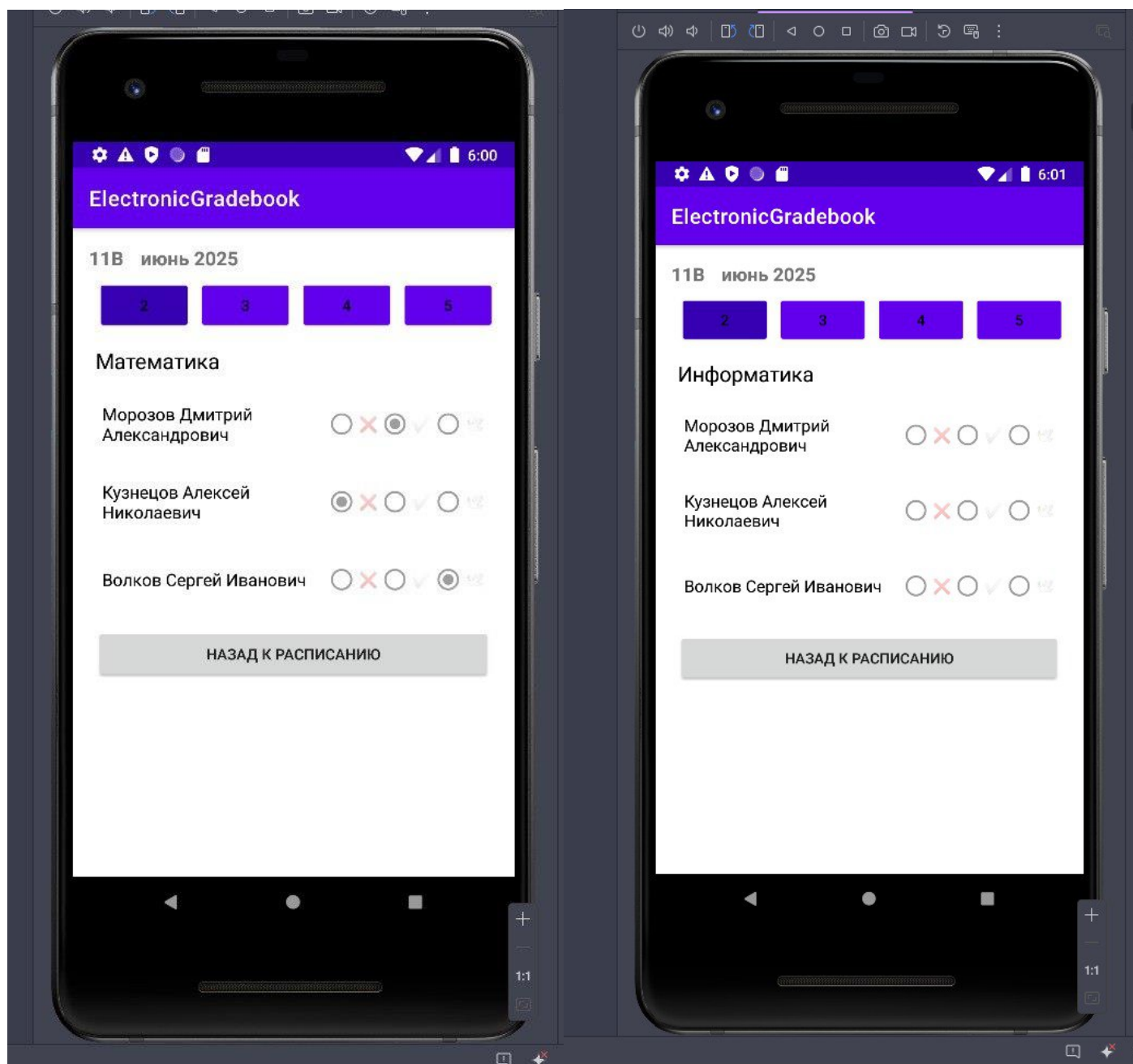


Рисунок 7 – Просмотр отметок посещаемости учащихся в электронном журнале

На рисунке 7 представлен интерфейс электронного журнала ElectronicGradebook, предназначенного для ведения посещаемости учащихся.

На изображении показаны два экрана: слева отображается предмет Математика, справа — Информатика.

В верхней части интерфейса указаны параметры текущего урока:

Учебный класс: 11В

Месяц и год: июнь 2025

Даты отображаются в виде кнопок с числами от 2 до 5, среди которых пользователь может выбрать нужную. Активная дата подсвечена более насыщенным фиолетовым цветом (например, 2 июня на левом экране, 5 июня — на правом).

Под заголовком предмета размещён список учеников:

Морозов Дмитрий Александрович

Кузнецов Алексей Николаевич

Волков Сергей Иванович

Для каждого ученика доступны четыре варианта отметки посещаемости:

× — ученик отсутствовал без уважительной причины;

З — ученик отсутствовал по уважительной причине;

✓ — ученик присутствовал на уроке;

○ — статус не выбран (отметка ещё не поставлена).

Педагог выбирает одну из опций для каждого ученика, фиксируя посещаемость на определённой дате и по конкретному предмету.

В нижней части экрана размещена кнопка НАЗАД К РАСПИСАНИЮ, позволяющая вернуться к просмотру общего расписания.

Таким образом, интерфейс предоставляет удобный способ быстрого и точного учета посещаемости учащихся, поддерживая различные причины отсутствия и облегчая работу преподавателя с журналом.

Возможности работы за админа



Рисунок 8 – Интерфейс управления группами за администратора

На рисунке 8 представлен интерфейс управления учебными группами системы. В верхней части расположен заголовок "Управление группами". Ниже последовательно перечислены группы: 11В (3 студентов), 11Б (3 студентов), 11А (3 студентов). Каждая группа отображается в отдельной строке с указанием количества учащихся.

Слева от списка групп находится кнопка "Добавить учеников в группы", позволяющая администратору включать новых учащихся в существующие группы. Справа расположена кнопка "Добавить новую группу" для создания дополнительных учебных групп.

Функционал доступен исключительно администраторам системы и предоставляет полный контроль над структурой учебных групп. Отображение точного количества студентов в каждой группе позволяет быстро оценить текущую загруженность.

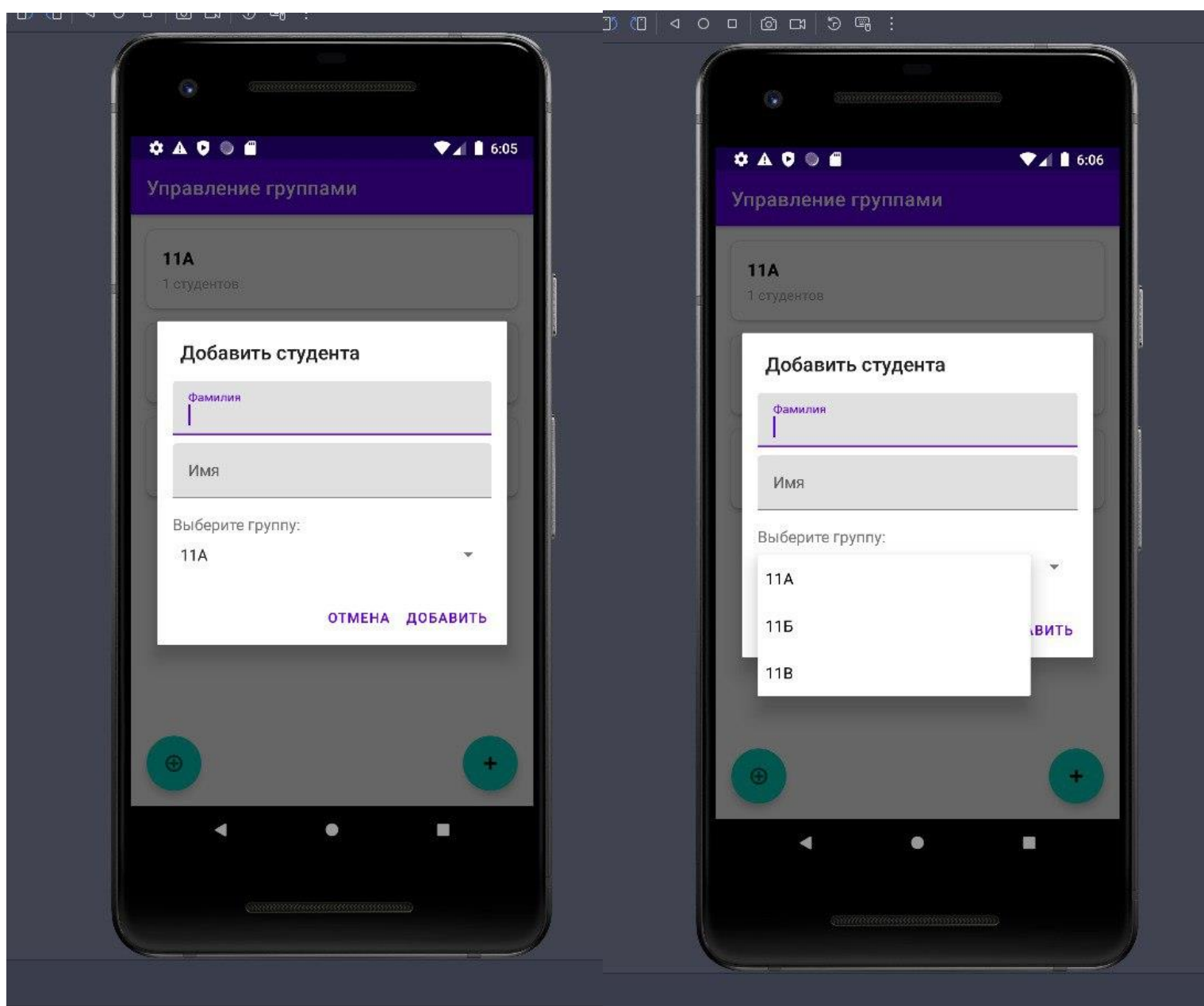


Рисунок 9 – Добавление нового студента в группу

На рисунке 9 показан интерфейс добавления нового студента в учебную группу. В верхней части экрана отображается заголовок "Управление группами" и название текущей группы "11А". Под ними расположена кнопка "Добавить студента".

При нажатии на кнопку открывается форма для ввода данных нового студента. В поле "Фамилия" уже введено значение "Высокая", поле "Имя" ожидает заполнения. Ниже находится выпадающий список для выбора группы с текущей группой "11А", разделителем "-" и доступными группами "11Б" и "11В".

В нижней части формы расположены две кнопки: "отмена" для отмены действия и добавить для подтверждения ввода нового студента. После заполнения всех полей и выбора группы администратор может завершить операцию добавления студента в систему.

ПРИЛОЖЕНИЕ

```
package com.example.electronicgradebook;

import android.app.AlertDialog;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.ArrayAdapter;
import android.widget.EditText;
import android.widget.Spinner;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.floatingactionbutton.FloatingActionButton;

import java.util.ArrayList;
import java.util.List;

public class AdminActivity extends AppCompatActivity {
    private static final String TAG = "AdminActivity";
    private RecyclerView recyclerViewGroups;
    private GroupAdapter groupAdapter;
    private List<Group> groups;
    private FloatingActionButton fabAddGroup;
    private FloatingActionButton fabAddStudent;
```

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_admin);

    // Инициализация UI компонентов
    recyclerViewGroups = findViewById(R.id.recyclerViewGroups);
    fabAddGroup = findViewById(R.id.fabAddGroup);
    fabAddStudent = findViewById(R.id.fabAddStudent);

    // Настройка RecyclerView
    recyclerViewGroups.setLayoutManager(new LinearLayoutManager(this));
    groups = new ArrayList<>(MockData.getGroups().values());
    groupAdapter = new GroupAdapter(groups);
    recyclerViewGroups.setAdapter(groupAdapter);

    // Обработчики нажатий
    fabAddGroup.setOnClickListener(v -> showAddGroupDialog());
    fabAddStudent.setOnClickListener(v -> showAddStudentDialog());

    Log.d(TAG, "AdminActivity created with " + groups.size() + "
groups");
}

private void showAddGroupDialog() {
    View dialogView =
LayoutInflater.from(this).inflate(R.layout.dialog_add_group, null);
    EditText editGroupName =
dialogView.findViewById(R.id.editGroupName);

    new AlertDialog.Builder(this)
        .setTitle("Добавить группу")
        .setView(dialogView)
        .setPositiveButton("Добавить", (dialog, which) -> {
            String groupName =
editGroupName.getText().toString().trim();
            if (!groupName.isEmpty()) {
                addNewGroup(groupName);
            } else {
                Toast.makeText(this, "Введите название группы",
Toast.LENGTH_SHORT).show();
            }
        })
        .setNegativeButton("Отмена", null)
        .show();
}

private void showAddStudentDialog() {
    View dialogView =
LayoutInflater.from(this).inflate(R.layout.dialog_add_student, null);
    EditText editLastName = dialogView.findViewById(R.id.editLastName);
    EditText editFirstName =
dialogView.findViewById(R.id.editFirstName);
    Spinner spinnerGroup = dialogView.findViewById(R.id.spinnerGroup);

    // Настройка спинера групп
    List<Group> groupsList = new
ArrayList<>(MockData.getGroups().values());
    ArrayAdapter<Group> adapter = new ArrayAdapter<>(this,
        android.R.layout.simple_spinner_item, groupsList);

    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_ite
m);
    spinnerGroup.setAdapter(adapter);
}

```

```

        new AlertDialog.Builder(this)
            .setTitle("Добавить студента")
            .setView(dialogView)
            .setPositiveButton("Добавить", (dialog, which) -> {
                String lastName = editLastName.getText().toString().trim();
                String firstName = editFirstName.getText().toString().trim();
                Group selectedGroup = (Group) spinnerGroup.getSelectedItem();

                if (!lastName.isEmpty() && !firstName.isEmpty() &&
                    selectedGroup != null) {
                    addNewStudent(lastName, firstName,
                        selectedGroup.getId());
                } else {
                    Toast.makeText(this, "Заполните все обязательные поля",
                        Toast.LENGTH_SHORT).show();
                }
            })
            .setNegativeButton("Отмена", null)
            .show();
    }

    private void addNewGroup(String groupName) {
        try {
            String groupId = "group" + (groups.size() + 1);
            Group newGroup = new Group(groupId, groupName);
            MockData.addGroup(newGroup);
            groups.add(newGroup);
            groupAdapter.notifyItemInserted(groups.size() - 1);
            Toast.makeText(this, "Группа добавлена",
                Toast.LENGTH_SHORT).show();
            Log.d(TAG, "Added new group: " + groupName);
        } catch (Exception e) {
            Log.e(TAG, "Error adding group", e);
            Toast.makeText(this, "Ошибка при добавлении группы",
                Toast.LENGTH_SHORT).show();
        }
    }

    private void addNewStudent(String lastName, String firstName, String
        groupId) {
        try {
            String studentId = "student" + System.currentTimeMillis();
            Student newStudent = new Student(studentId, lastName, firstName,
                groupId);
            MockData.addStudent(newStudent);
            Toast.makeText(this, "Студент добавлен",
                Toast.LENGTH_SHORT).show();
            Log.d(TAG, "Added new student: " + lastName + " " + firstName);
        } catch (Exception e) {
            Log.e(TAG, "Error adding student", e);
            Toast.makeText(this, "Ошибка при добавлении студента",
                Toast.LENGTH_SHORT).show();
        }
    }
}

```

```

package com.example.electronicgradebook;

import android.app.Application;
import android.util.Log;
import java.util.List;
import java.util.Map;
import com.google.firebase.FirebaseApp;
import com.google.android.gms.common.GoogleApiAvailability;

public class ElectronicGradebookApp extends Application {
    private static final String TAG = "ElectronicGradebookApp";

    @Override
    public void onCreate() {
        super.onCreate();
        Log.d(TAG, "Application onCreate started");

        try {
            // Инициализация Firebase
            if (FirebaseApp.getApps(this).isEmpty()) {
                FirebaseApp.initializeApp(this);
                Log.d(TAG, "Firebase initialized successfully");
            }

            // Проверка доступности Google Play Services
            GoogleApiAvailability googleAPI =
GoogleApiAvailability.getInstance();
            int resultCode = googleAPI.isGooglePlayServicesAvailable(this);
            if (resultCode !=
com.google.android.gms.common.ConnectionResult.SUCCESS) {
                Log.e(TAG, "Google Play Services not available: " +
googleAPI.getErrorString(resultCode));
            } else {
                Log.d(TAG, "Google Play Services available");
            }

            Log.d(TAG, "Starting MockData initialization");
            MockData.init(this);
            MockData.syncWithFirebase();
            Log.d(TAG, "MockData initialized successfully");

            // Проверяем, что данные загружены
            Map<String, Group> groups = MockData.getGroups();
            Log.d(TAG, "Loaded " + groups.size() + " groups after
initialization");

            for (Group group : groups.values()) {
                List<Student> students =
MockData.getStudentsByGroup(group.getId());
                Log.d(TAG, "Group " + group.getName() + " has " +
students.size() + " students");
                for (Student student : students) {
                    Log.d(TAG, "Student: " + student.getFullName() + " in
group " + group.getName());
                }
            }
        } catch (Exception e) {
            Log.e(TAG, "Error initializing app", e);
        }
    }
}

```



```

package com.example.electronicgradebook;

public class Group {
    private String id;
    private String name;

    public Group(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    @Override
    public String toString() {
        return name;
    }
}

```

Листинг 1 Group

```

package com.example.electronicgradebook;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import java.util.List;

public class GroupAdapter extends
RecyclerView.Adapter<GroupAdapter.GroupViewHolder> {
    private final List<Group> groups;

    public GroupAdapter(List<Group> groups) {
        this.groups = groups;
    }

    @NonNull
    @Override
    public GroupViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int
viewType) {
        View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_group, parent, false);
        return new GroupViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull GroupViewHolder holder, int
position) {
        Group group = groups.get(position);
        holder.textGroupName.setText(group.getName());
    }
}

```

```

        // Получаем количество студентов в группе
        int studentCount = 0;
        for (Student student : MockData.getStudents().values()) {
            if (student.getGroupId().equals(group.getId())) {
                studentCount++;
            }
        }
        holder.textStudentCount.setText(studentCount + " студентов");
    }

    @Override
    public int getItemCount() {
        return groups.size();
    }

    static class GroupViewHolder extends RecyclerView.ViewHolder {
        TextView textGroupName;
        TextView textStudentCount;

        GroupViewHolder(View itemView) {
            super(itemView);
            textGroupName = itemView.findViewById(R.id.textGroupName);
            textStudentCount = itemView.findViewById(R.id.textStudentCount);
        }
    }
}

```

Листинг 2 GroupAdapter

```

package com.example.electronicgradebook;

import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ProgressBar;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import com.google.firebase.auth.AuthResult;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.firestore.FirebaseFirestore;

public class LoginActivity extends AppCompatActivity {
    private EditText editEmail, editPassword;
    private Button btnLogin, btnRegister;
    private ProgressBar progressBar;
    private FirebaseAuth mAuth;
    private static final String TAG = "LoginActivity";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
    }
}

```

```

setContentView(R.layout.activity_login);

editEmail = findViewById(R.id.editEmail);
editPassword = findViewById(R.id.editPassword);
btnLogin = findViewById(R.id.btnLogin);
btnRegister = findViewById(R.id.btnRegister);
progressBar = findViewById(R.id.progressBar);
mAuth = FirebaseAuth.getInstance();

btnLogin.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        loginUser();
    }
});

btnRegister.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(LoginActivity.this,
RegisterActivity.class);
        startActivity(intent);
    }
});

}

private void loginUser() {
    String email = editEmail.getText().toString().trim();
    String password = editPassword.getText().toString().trim();

    if (TextUtils.isEmpty(email) || TextUtils.isEmpty(password)) {
        Toast.makeText(this, "Введите email и пароль",
Toast.LENGTH_SHORT).show();
        return;
    }

    progressBar.setVisibility(View.VISIBLE);
    mAuth.signInWithEmailAndPassword(email, password)
        .addOnCompleteListener(this, new
OnCompleteListener<AuthResult>() {
        @Override
        public void onComplete(@NonNull Task<AuthResult> task) {
            progressBar.setVisibility(View.GONE);
            if (task.isSuccessful()) {
                FirebaseUser user = mAuth.getCurrentUser();
                if (user != null) {
                    String userId = user.getId();
                    FirebaseFirestore db =
FirebaseFirestore.getInstance();
                    db.collection("users").document(userId).get()
                        .addOnSuccessListener(documentSnapshot -> {
                            if (documentSnapshot.exists()) {
                                String role =
documentSnapshot.getString("role");

                                Intent intent;
                                if ("student".equals(role)) {
                                    intent = new
Intent(LoginActivity.this, StudentActivity.class);
                                    intent.putExtra("STUDENT_ID",
userId);
                                } else if ("teacher".equals(role)) {
                                    intent = new
Intent(LoginActivity.this, TeacherActivity.class);
                                    intent.putExtra("TEACHER_ID",
userId);
                                }
                            }
                        })
                    ;
                }
            }
        }
    });
}

```

```

                                } else if ("admin".equals(role)) {
                                    intent = new
Intent(LoginActivity.this, AdminActivity.class);
                                    intent.putExtra("ADMIN_ID",
userId);
                                } else {

Toast.makeText(LoginActivity.this, "Неизвестная роль пользователя",
Toast.LENGTH_SHORT).show();

                                    return;
                                }
                                startActivity(intent);
                                finish();
                                } else {
                                    Toast.makeText(LoginActivity.this,
"Профиль пользователя не найден", Toast.LENGTH_SHORT).show();
                                }
                            })
                            .addOnFailureListener(e -> {
                                Toast.makeText(LoginActivity.this,
"Ошибка загрузки профиля: " + e.getMessage(), Toast.LENGTH_SHORT).show();
                            });
                        }
                    } else {
                        Toast.makeText(LoginActivity.this, "Ошибка входа: "
+ task.getException().getMessage(), Toast.LENGTH_SHORT).show();
                    }
                }
            });
        }
    }
}

```

Листинг 3 LoginActivity

```

package com.example.electronicgradebook;

import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.HorizontalScrollView;
import android.widget.LinearLayout;
import android.widget.RadioGroup;
import android.widget.Spinner;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Date;
import java.util.HashSet;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import com.example.electronicgradebook.Group;
import com.example.electronicgradebook.Student;

public class MarkStudentsActivity extends AppCompatActivity {
    private static final String TAG = "MarkStudentsActivity";
}

```

```

private Spinner spinnerGroup;
private Spinner spinnerSubject;
private LinearLayout studentListContainer;
private TextView textClassInfo;
private TextView textMonthYear;
private String selectedDate;
private String selectedGroupId;
private String selectedSubject;
private List<Button> dateButtons = new ArrayList<>();

private static final SimpleDateFormat DATE_FORMAT = new
SimpleDateFormat("yyyy-MM-dd", Locale.getDefault());
private static final SimpleDateFormat MONTH_YEAR_FORMAT = new
SimpleDateFormat("LLLL yyyy", new Locale("ru"));
private static final SimpleDateFormat DAY_FORMAT = new
SimpleDateFormat("d", Locale.getDefault());

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_mark_students);

    Log.d(TAG, "onCreate started");

    // Инициализация views
    spinnerGroup = findViewById(R.id.spinnerGroup);
    spinnerSubject = findViewById(R.id.spinnerSubject);
    studentListContainer = findViewById(R.id.studentListContainer);
    textClassInfo = findViewById(R.id.textClassInfo);
    textMonthYear = findViewById(R.id.textMonthYear);

    // Загружаем список групп
    loadGroups();

    // Устанавливаем текущий месяц и год
    textMonthYear.setText(MONTH_YEAR_FORMAT.format(new Date()));

    // Генерируем кнопки с датами
    generateDateButtons();

    // Обработчик выбора группы
    spinnerGroup.setOnItemClickListener(new
AdapterView.OnItemClickListener() {
        @Override
        public void onItemClick(AdapterView<?> parent, View view, int
position, long id) {
            Group selectedGroup = (Group)
parent.getItemAtPosition(position);
            selectedGroupId = selectedGroup.getId();
            Log.d(TAG, "Selected group: " + selectedGroup.getName() + "
(ID: " + selectedGroupId + ")");
            updateSubjects();
        }

        @Override
        public void onNothingSelected(AdapterView<?> parent) {
            selectedGroupId = null;
            spinnerSubject.setAdapter(null);
        }
    });

    // Обработчик выбора предмета
    spinnerSubject.setOnItemClickListener(new
AdapterView.OnItemClickListener() {

```

```

        @Override
        public void onItemClick(AdapterView<?> parent, View view, int
position, long id) {
            selectedSubject = (String)
parent.getItemAtPosition(position);
            Log.d(TAG, "Selected subject: " + selectedSubject);
            if (selectedDate != null) {
                loadStudents();
            }
        }

        @Override
        public void onNothingSelected(AdapterView<?> parent) {
            selectedSubject = null;
            studentListContainer.removeAllViews();
        }
    });
}

private void generateDateButtons() {
    LinearLayout datesContainer = findViewById(R.id.datesContainer);
    datesContainer.removeAllViews();
    dateButtons.clear();

    // Создаем кнопки для каждого дня сентября 2024
    Calendar calendar = Calendar.getInstance();
    calendar.set(2024, Calendar.SEPTEMBER, 1);

    // Создаем контейнер для горизонтальной прокрутки
    HorizontalScrollView scrollView = new HorizontalScrollView(this);
    LinearLayout.LayoutParams scrollParams = new
LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT,
        LinearLayout.LayoutParams.WRAP_CONTENT
    );
    scrollView.setLayoutParams(scrollParams);

    // Создаем контейнер для кнопок
    LinearLayout buttonsContainer = new LinearLayout(this);
    buttonsContainer.setOrientation(LinearLayout.HORIZONTAL);
    buttonsContainer.setLayoutParams(new LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.WRAP_CONTENT,
        LinearLayout.LayoutParams.WRAP_CONTENT
    ));

    // Добавляем кнопки для каждого дня
    while (calendar.get(Calendar.MONTH) == Calendar.SEPTEMBER) {
        // Пропускаем выходные
        int dayOfWeek = calendar.get(Calendar.DAY_OF_WEEK);
        if (dayOfWeek != Calendar.SATURDAY && dayOfWeek !=
Calendar.SUNDAY) {
            Button dateButton = new Button(this);
            dateButton.setText(DAY_FORMAT.format(calendar.getTime()));

            // Сохраняем дату в миллисекундах в теге кнопки
            dateButton.setTag(calendar.getTimeInMillis());

            // Устанавливаем обработчик нажатия
            dateButton.setOnClickListener(v -> {
                // Получаем дату из тега кнопки
                long dateInMillis = (long) v.getTag();
                Calendar cal = Calendar.getInstance();
                cal.setTimeInMillis(dateInMillis);
                selectedDate = DATE_FORMAT.format(cal.getTime());
            });
        }
        dateButtons.addView(dateButton);
        calendar.add(Calendar.DAY_OF_MONTH, 1);
    }
}

```

```

        // Обновляем UI
        for (Button btn : dateButtons) {

btn.setBackgroundTintList (ContextCompat.getColorStateList (this,
R.color.purple_500));
        }

dateButton.setBackgroundTintList (ContextCompat.getColorStateList (this,
R.color.purple_700));

        // Загружаем список студентов
        if (selectedSubject != null) {
            loadStudents ();
        }
    });

    // Добавляем кнопку в контейнер
    LinearLayout.LayoutParams buttonParams = new
LinearLayout.LayoutParams (
        LinearLayout.LayoutParams.WRAP_CONTENT,
        LinearLayout.LayoutParams.WRAP_CONTENT
    );
    buttonParams.setMargins (8, 0, 8, 0);
    dateButton.setLayoutParams (buttonParams);
    buttonsContainer.addView (dateButton);
    dateButtons.add (dateButton);
}

// Переходим к следующему дню
calendar.add (Calendar.DAY_OF_MONTH, 1);
}

// Добавляем контейнер с кнопками в ScrollView
scrollView.addView (buttonsContainer);
datesContainer.addView (scrollView);

// Выбираем первую кнопку по умолчанию
if (!dateButtons.isEmpty()) {
    dateButtons.get (0).performClick ();
}
}

private void loadGroups () {
    Map<String, Group> groupsMap = MockData.getGroups ();
    List<Group> uniqueGroups = new ArrayList<> ();
    Set<String> seenNames = new HashSet<> ();
    for (Group group : groupsMap.values ()) {
        String name = group.getName ().trim ().toLowerCase ();
        if (!seenNames.contains (name)) {
            uniqueGroups.add (group);
            seenNames.add (name);
        }
    }
    ArrayAdapter<Group> adapter = new ArrayAdapter<> (this,
        android.R.layout.simple_spinner_item, uniqueGroups);

adapter.setDropDownViewResource (android.R.layout.simple_spinner_dropdown_ite
m);
    spinnerGroup.setAdapter (adapter);
}

private void updateSubjects () {
    if (selectedGroupId == null) {

```

```

        spinnerSubject.setAdapter(null);
        return;
    }

    try {
        // Получаем день недели для выбранной даты
        Calendar calendar = Calendar.getInstance();
        calendar.setTime(DateFormat.parse(selectedDate));

        // Получаем день недели на русском языке
        String[] daysOfWeek = {"Воскресенье", "Понедельник", "Вторник",
"Среда", "Четверг", "Пятница", "Суббота"};
        String dayOfWeek = daysOfWeek[calendar.get(Calendar.DAY_OF_WEEK)
- 1];

        Log.d(TAG, "Day of week: " + dayOfWeek);

        // Получаем предметы на выбранный день
        List<String> subjects =
MockData.getSubjectsForGroupAndDay(selectedGroupId, dayOfWeek);
        Log.d(TAG, "Found " + subjects.size() + " subjects for group " +
selectedGroupId + " on " + dayOfWeek);

        ArrayAdapter<String> subjectAdapter = new ArrayAdapter<>(this,
            android.R.layout.simple_spinner_item, subjects);

subjectAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropd
own_item);
        spinnerSubject.setAdapter(subjectAdapter);

        // Обновляем информацию о классе
        Group selectedGroup = null;
        for (Group group : MockData.getGroups().values()) {
            if (group.getId().equals(selectedGroupId)) {
                selectedGroup = group;
                break;
            }
        }
        if (selectedGroup != null) {
            textClassInfo.setText(selectedGroup.getName());
        }

    } catch (Exception e) {
        Log.e(TAG, "Error updating subjects", e);
        Toast.makeText(this, "Ошибка при загрузке предметов",
Toast.LENGTH_SHORT).show();
    }
}

private void loadStudents() {
    if (selectedGroupId == null || selectedSubject == null ||
selectedDate == null) {
        Log.d(TAG, "Cannot load students: missing required data");
        return;
    }

    Log.d(TAG, "Loading students for group: " + selectedGroupId + ",
subject: " + selectedSubject + ", date: " + selectedDate);
    studentListContainer.removeAllViews();

    try {
        // Получаем список студентов группы
        String normGroup = MockData.normalizeGroupName(selectedGroupId);
        List<Student> students = MockData.getStudentsByGroup(normGroup);
        Log.d(TAG, "Found " + students.size() + " students in group");
    }
}

```



```

        if (students.isEmpty()) {
            TextView noStudentsText = new TextView(this);
            noStudentsText.setText("В группе нет студентов");
            noStudentsText.setTextSize(16);
            noStudentsText.setPadding(16, 16, 16, 16);
            studentListContainer.addView(noStudentsText);
            return;
        }

        for (Student student : students) {
            View studentView =
                getLayoutInflater().inflate(R.layout.item_student_attendance,
                    studentListContainer, false);
            TextView studentNameText =
                studentView.findViewById(R.id.textStudentName);
            RadioGroup attendanceGroup =
                studentView.findViewById(R.id.radioGroupAttendance);

            if (studentNameText == null || attendanceGroup == null) {
                Log.e(TAG, "Could not find views in student attendance
layout");
                continue;
            }

            studentNameText.setText(student.getFullName());

            // Получаем сохраненную отметку
            String savedMark = MockData.getAttendance(selectedGroupId,
                selectedDate, selectedSubject, student.getId());
            Log.d(TAG, "Student " + student.getFullName() + "
attendance: " + savedMark);

            // Устанавливаем сохраненную отметку, если она есть
            if (savedMark != null) {
                switch (savedMark) {
                    case "present":
                        attendanceGroup.check(R.id.radioPresent);
                        break;
                    case "absent":
                        attendanceGroup.check(R.id.radioAbsent);
                        break;
                    case "excused":
                        attendanceGroup.check(R.id.radioExcused);
                        break;
                }
            }

            // Добавляем обработчик изменения отметки
            attendanceGroup.setOnCheckedChangeListener((group,
checkedId) -> {
                String mark;
                if (checkedId == R.id.radioPresent) {
                    mark = "present";
                } else if (checkedId == R.id.radioAbsent) {
                    mark = "absent";
                } else if (checkedId == R.id.radioExcused) {
                    mark = "excused";
                } else {
                    return;
                }

                // Сохраняем отметку
                MockData.saveAttendance(selectedGroupId, selectedDate,

```

```

selectedSubject, student.getId(), mark);
        Log.d(TAG, "Saved attendance for " +
student.getFullName() + ": " + mark);
    });

    studentListContainer.addView(studentView);
    Log.d(TAG, "Added student to container: " +
student.getFullName());
    }
    } catch (Exception e) {
        Log.e(TAG, "Error loading students", e);
        Toast.makeText(this, "Ошибка при загрузке списка студентов",
Toast.LENGTH_SHORT).show();
    }
    }
}

```

Листинг 4 MarkStudentsActivity

```

package com.example.electronicgradebook;

import android.content.Context;
import android.content.SharedPreferences;
import android.util.Log;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import com.example.electronicgradebook.Group;
import com.example.electronicgradebook.Student;
import org.json.JSONException;
import org.json.JSONObject;
import java.util.Iterator;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.DocumentSnapshot;

public class MockData {
    private static final String TAG = "MockData";
    private static final String[] DAYS_OF_WEEK = {"Понедельник", "Вторник",
"Среда", "Четверг", "Пятница", "Суббота"};
    private static Context appContext;
    private static boolean isInitialized = false;
    private static Map<String, Group> groups = new HashMap<>();
    private static Map<String, Student> students = new HashMap<>();
    private static Map<String, Map<String, List<String>>> schedule = new
HashMap<>();
    private static Map<String, String> attendanceMarks = new HashMap<>();
    private static final String PREFS_NAME = "ElectronicGradebookPrefs";
    private static final String KEY_STUDENTS = "students";
    private static final String KEY_GROUPS = "groups";

    public static void init(Context context) {
        if (isInitialized) {
            Log.d(TAG, "MockData already initialized");
            return;
        }
        Log.d(TAG, "Initializing MockData");
        appContext = context.getApplicationContext();
        loadSavedData();
        // Если нет сохранённых данных, инициализируем начальные
        if (groups.isEmpty()) {

```

```

        Log.d(TAG, "No saved groups found, initializing default
groups");
        addDefaultGroup("11A");
        addDefaultGroup("11B");
        addDefaultGroup("11B");
        saveGroups();
    } else {
        Log.d(TAG, "Loaded " + groups.size() + " groups from storage");
    }
    if (students.isEmpty()) {
        Log.d(TAG, "No saved students found, initializing default
students");
        students.put("student1", new Student("student1", "Иванов",
"Иван", "11A"));
        students.put("student2", new Student("student2", "Петров",
"Петр", "11A"));
        students.put("student3", new Student("student3", "Сидоров",
"Сидор", "11A"));
        students.put("student4", new Student("student4", "Смирнова",
"Анна", "11B"));
        students.put("student5", new Student("student5", "Козлова",
"Мария", "11B"));
        students.put("student6", new Student("student6", "Новикова",
"Елена", "11B"));
        students.put("student7", new Student("student7", "Кузнецов",
"Алексей", "11B"));
        students.put("student8", new Student("student8", "Морозов",
"Дмитрий", "11B"));
        students.put("student9", new Student("student9", "Волков",
"Сергей", "11B"));
        saveStudents();
    } else {
        Log.d(TAG, "Loaded " + students.size() + " students from
storage");
    }
    // Инициализация расписания
    for (String groupName : groups.keySet()) {
        schedule.put(groupName, new HashMap<>());
        Map<String, List<String>> groupSchedule =
schedule.get(groupName);
        groupSchedule.put("Понедельник", Arrays.asList("Математика",
"Физика", "Информатика", "Русский язык", "Литература"));
        groupSchedule.put("Вторник", Arrays.asList("Английский язык",
"История", "Обществознание", "Биология", "Химия"));
        groupSchedule.put("Среда", Arrays.asList("Математика", "Физика",
"Информатика", "Русский язык", "Литература"));
        groupSchedule.put("Четверг", Arrays.asList("Английский язык",
"История", "Обществознание", "Биология", "Химия"));
        groupSchedule.put("Пятница", Arrays.asList("Математика",
"Физика", "Информатика", "Русский язык", "Литература"));
        groupSchedule.put("Суббота", Arrays.asList("Английский язык",
"История", "Обществознание", "Биология", "Химия"));
    }
    loadAttendanceMarks();
    isInitialized = true;
    Log.d(TAG, "MockData initialized successfully");
}

private static void checkInitialization() {
    if (!isInitialized) {
        throw new IllegalStateException("MockData not initialized. Call
MockData.init() first.");
    }
}
}

```

```

    public static Map<String, Group> getGroups() {
        checkInitialization();
        Map<String, Group> normalizedGroups = new HashMap<>();
        for (Group group : groups.values()) {
            String normName = normalizeGroupName(group.getName());
            normalizedGroups.put(normName, new Group(normName, normName));
        }
        return normalizedGroups;
    }

    public static List<String> getSubjectsForGroupAndDay(String groupName,
String dayOfWeek) {
        checkInitialization();
        Map<String, List<String>> groupSchedule = schedule.get(groupName);
        if (groupSchedule == null) {
            return new ArrayList<>();
        }
        List<String> subjects = groupSchedule.get(dayOfWeek);
        return subjects != null ? new ArrayList<>(subjects) : new
ArrayList<>();
    }

    public static List<Student> getStudentsByGroup(String groupName) {
        checkInitialization();
        String normGroup = normalizeGroupName(groupName);
        List<Student> groupStudents = new ArrayList<>();
        for (Student student : students.values()) {
            if (normalizeGroupName(student.getGroupId()).equals(normGroup))
{
                groupStudents.add(student);
            }
        }
        return groupStudents;
    }

    private static void loadAttendanceMarks() {
        if (appContext == null) {
            Log.e(TAG, "Cannot load attendance marks: appContext is null");
            return;
        }

        SharedPreferences prefs =
appContext.getSharedPreferences("attendance_marks", Context.MODE_PRIVATE);
        String marksString = prefs.getString("marks", "");
        Log.d(TAG, "Loading attendance marks: " + (marksString.isEmpty() ?
"no saved marks" : marksString));

        attendanceMarks.clear();
        if (!marksString.isEmpty()) {
            String[] marks = marksString.split(";");
            for (String mark : marks) {
                if (mark.isEmpty()) continue;
                String[] parts = mark.split("=");
                if (parts.length == 2) {
                    attendanceMarks.put(parts[0], parts[1]);
                    Log.d(TAG, "Loaded mark: " + parts[0] + " -> " +
parts[1]);
                }
            }
        }
        Log.d(TAG, "Loaded " + attendanceMarks.size() + " attendance
marks");
    }

```

```

        public static void saveAttendance(String groupName, String date, String
subject, String studentId, String status) {
            checkInitialization();
            String key = groupName + ":" + date + ":" + subject + ":" +
studentId;
            Log.d(TAG, "Saving attendance: " + key + " -> " + status);
            attendanceMarks.put(key, status);
            saveAttendanceMarks();
        }

        public static String getAttendance(String groupName, String date, String
subject, String studentId) {
            checkInitialization();
            String key = groupName + ":" + date + ":" + subject + ":" +
studentId;
            String status = attendanceMarks.get(key);
            Log.d(TAG, "Getting attendance: " + key + " -> " + status);
            return status;
        }

        private static void saveAttendanceMarks() {
            if (appContext == null) {
                Log.e(TAG, "Cannot save attendance marks: appContext is null");
                return;
            }

            SharedPreferences prefs =
appContext.getSharedPreferences("attendance_marks", Context.MODE_PRIVATE);
            StringBuilder sb = new StringBuilder();

            for (Map.Entry<String, String> entry : attendanceMarks.entrySet()) {
                if (sb.length() > 0) {
                    sb.append(";");
                }
                sb.append(entry.getKey()).append("=").append(entry.getValue());
            }

            String marksString = sb.toString();
            Log.d(TAG, "Saving attendance marks: " + (marksString.isEmpty() ?
"no marks to save" : marksString));

            prefs.edit().putString("marks", marksString).apply();
            Log.d(TAG, "Saved " + attendanceMarks.size() + " attendance marks");
        }

        public static void addGroup(Group group) {
            if (group != null && group.getName() != null) {
                String normName = normalizeGroupName(group.getName());
                groups.put(normName, new Group(normName, normName));
                schedule.put(normName, new HashMap<>());
                for (String day : DAYS_OF_WEEK) {
                    schedule.get(normName).put(day, new ArrayList<>());
                }
                saveGroups();
                Log.d(TAG, "Added new group: " + normName);
            }
        }

        public static void addStudent(Student student) {
            if (student != null && student.getId() != null) {
                String normGroup = normalizeGroupName(student.getGroupId());
                students.put(student.getId(), new Student(student.getId(),
student.getLastName(), student.getFirstName(), normGroup));
            }
        }

```

```

        saveStudents();
        Log.d(TAG, "Added new student: " + student.getLastName() + " " +
student.getFirstName());
    }

}

public static Map<String, Student> getStudents() {
    return students;
}

private static void loadSavedData() {
    if (appContext == null) return;

    SharedPreferences prefs =
appContext.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);

    // Загружаем группы
    String groupsJson = prefs.getString(KEY_GROUPS, "");
    if (!groupsJson.isEmpty()) {
        try {
            JSONObject json = new JSONObject(groupsJson);
            Iterator<String> keys = json.keys();
            while (keys.hasNext()) {
                String key = keys.next();
                JSONObject groupJson = json.getJSONObject(key);
                groups.put(key, new Group(key,
groupJson.getString("name")));
            }
        } catch (JSONException e) {
            Log.e(TAG, "Error loading groups", e);
        }
    }

    // Загружаем студентов
    String studentsJson = prefs.getString(KEY_STUDENTS, "");
    if (!studentsJson.isEmpty()) {
        try {
            JSONObject json = new JSONObject(studentsJson);
            Iterator<String> keys = json.keys();
            while (keys.hasNext()) {
                String key = keys.next();
                JSONObject studentJson = json.getJSONObject(key);
                students.put(key, new Student(
                    key,
                    studentJson.getString("lastName"),
                    studentJson.getString("firstName"),
                    studentJson.getString("groupId")
                ));
            }
        } catch (JSONException e) {
            Log.e(TAG, "Error loading students", e);
        }
    }
}

private static void saveGroups() {
    if (appContext == null) return;

    try {
        JSONObject json = new JSONObject();
        for (Map.Entry<String, Group> entry : groups.entrySet()) {
            JSONObject groupJson = new JSONObject();
            groupJson.put("name", entry.getValue().getName());
            json.put(entry.getKey(), groupJson);
        }
    }
}

```

```

    }

    SharedPreferences prefs =
appContext.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
    prefs.edit().putString(KEY_GROUPS, json.toString()).apply();
    } catch (JSONException e) {
        Log.e(TAG, "Error saving groups", e);
    }
}

private static void saveStudents() {
    if (appContext == null) return;

    try {
        JSONObject json = new JSONObject();
        for (Map.Entry<String, Student> entry : students.entrySet()) {
            JSONObject studentJson = new JSONObject();
            studentJson.put("lastName", entry.getValue().getLastName());
            studentJson.put("firstName",
entry.getValue().getFirstName());
            studentJson.put("groupId", entry.getValue().getGroupId());
            json.put(entry.getKey(), studentJson);
        }

        SharedPreferences prefs =
appContext.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
        prefs.edit().putString(KEY_STUDENTS, json.toString()).apply();
    } catch (JSONException e) {
        Log.e(TAG, "Error saving students", e);
    }
}

public static void deleteStudent(String studentId) {
    if (students.containsKey(studentId)) {
        students.remove(studentId);
        saveStudents();
        Log.d(TAG, "Deleted student: " + studentId);
    }
}

public static void deleteGroup(String groupName) {
    if (groups.containsKey(groupName)) {
        groups.remove(groupName);
        schedule.remove(groupName);
        List<String> studentsToRemove = new ArrayList<>();
        for (Map.Entry<String, Student> entry : students.entrySet()) {
            if (entry.getValue().getGroupId().equals(groupName)) {
                studentsToRemove.add(entry.getKey());
            }
        }
        for (String studentId : studentsToRemove) {
            students.remove(studentId);
        }
        List<String> marksToRemove = new ArrayList<>();
        for (String key : attendanceMarks.keySet()) {
            if (key.startsWith(groupName + ":")) {
                marksToRemove.add(key);
            }
        }
        for (String key : marksToRemove) {
            attendanceMarks.remove(key);
        }
        saveGroups();
        saveStudents();
    }
}

```

```

        saveAttendanceMarks();
        Log.d(TAG, "Deleted group: " + groupName + " and all related
data");
    }
}

public static void syncWithFirebase() {
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection("users")
        .whereEqualTo("role", "student")
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            for (DocumentSnapshot doc : queryDocumentSnapshots) {
                String id = doc.getId();
                String lastName = doc.getString("lastName");
                String firstName = doc.getString("firstName");
                String groupName = doc.getString("group");
                Log.d(TAG, "Firebase student: " + lastName + " " +
firstName + " group: '" + groupName + "'");
                if (groupName != null) {
                    String normGroup = normalizeGroupName(groupName);
                    if (!groups.containsKey(normGroup)) {
                        Group group = new Group(groupName, groupName);
                        addGroup(group);
                        Log.d(TAG, "Added group from Firebase: '" +
groupName + "' (normalized: '" + normGroup + "')");
                    }
                }
                if (lastName != null && firstName != null && groupName
!= null) {
                    if (!students.containsKey(id)) {
                        Student student = new Student(id, lastName,
firstName, groupName);
                        addStudent(student);
                        Log.d(TAG, "Added student: " + lastName + " " +
firstName + " to group: '" + groupName + "' (normalized: '" +
normalizeGroupName(groupName) + "')");
                    }
                }
            }
        });
}

private static void addDefaultGroup(String groupName) {
    for (String key : groups.keySet()) {
        if (key.trim().equalsIgnoreCase(groupName.trim())) return;
    }
    groups.put(groupName, new Group(groupName, groupName));
}

public static String normalizeGroupName(String name) {
    if (name == null) return "";
    // Заменяем латинские буквы на кириллические
    name = name.replace('A', 'А').replace('B', 'В').replace('V', 'В');
    name = name.replace('a', 'а').replace('b', 'в').replace('v', 'в');
    // Удаляем пробелы и приводим к верхнему регистру
    return name.trim().toUpperCase();
}
}

```

Листинг 5 MockData

```
package com.example.electronicgradebook;
```



```

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Spinner;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;
import android.widget.AdapterView;
import android.widget.TextView;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class RegisterActivity extends AppCompatActivity {
    private EditText editFirstName, editLastName, editEmail, editPassword;
    private Spinner spinnerRole, spinnerGroup;
    private Button btnRegister;
    private FirebaseAuth mAuth;
    private FirebaseFirestore db;
    private List<String> groupList = new ArrayList<>();
    private ArrayAdapter<String> groupAdapter;
    private TextView groupLabel;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_register);

        mAuth = FirebaseAuth.getInstance();
        db = FirebaseFirestore.getInstance();

        editFirstName = findViewById(R.id.editFirstName);
        editLastName = findViewById(R.id.editLastName);
        editEmail = findViewById(R.id.editEmail);
        editPassword = findViewById(R.id.editPassword);
        spinnerRole = findViewById(R.id.spinnerRole);
        spinnerGroup = findViewById(R.id.spinnerGroup);
        btnRegister = findViewById(R.id.btnRegister);
        groupLabel = findViewById(R.id.groupLabel);

        // Настройка спиннера ролей
        ArrayAdapter<CharSequence> roleAdapter =
        ArrayAdapter.createFromResource(this,
            R.array.roles_array, android.R.layout.simple_spinner_item);
        roleAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
        spinnerRole.setAdapter(roleAdapter);

        // Настройка спиннера групп
        groupAdapter = new ArrayAdapter<>(this,
        android.R.layout.simple_spinner_item, groupList);
        groupAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
        spinnerGroup.setAdapter(groupAdapter);
        loadGroupsFromFirestore();
    }

```

```

        spinnerRole.setOnItemSelectedListener(new
AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int
position, long id) {
        String role = spinnerRole.getSelectedItem().toString();
        if (role.equals("Ученик")) {
            groupLabel.setVisibility(View.VISIBLE);
            spinnerGroup.setVisibility(View.VISIBLE);
        } else {
            groupLabel.setVisibility(View.GONE);
            spinnerGroup.setVisibility(View.GONE);
        }
    }
    @Override
    public void onNothingSelected(AdapterView<?> parent) {}
});

btnRegister.setOnClickListener(v -> {
    String firstName = editFirstName.getText().toString().trim();
    String lastName = editLastName.getText().toString().trim();
    String email = editEmail.getText().toString().trim();
    String password = editPassword.getText().toString().trim();
    String role = spinnerRole.getSelectedItem().toString();
    String group = spinnerGroup.getSelectedItem() != null ?
spinnerGroup.getSelectedItem().toString() : "";
    if (role.equals("Учитель")) role = "teacher";
    else if (role.equals("Ученик")) role = "student";
    else if (role.equals("Администратор")) role = "admin";
    final String finalRole = role;

    if (firstName.isEmpty() || lastName.isEmpty() || email.isEmpty()
|| password.isEmpty() || group.isEmpty()) {
        Toast.makeText(this, "Пожалуйста, заполните все поля и
выберите группу", Toast.LENGTH_SHORT).show();
        return;
    }

    mAuth.createUserWithEmailAndPassword(email, password)
        .addOnCompleteListener(this, task -> {
            if (task.isSuccessful()) {
                String userId = mAuth.getCurrentUser().getUid();
                Map<String, Object> userMap = new HashMap<>();
                userMap.put("firstName", firstName);
                userMap.put("lastName", lastName);
                userMap.put("email", email);
                userMap.put("role", finalRole);
                userMap.put("group", group);
                userMap.put("id", userId);

                db.collection("users").document(userId).set(userMap)
                    .addOnSuccessListener(aVoid -> {
                        Toast.makeText(this, "Регистрация
успешна", Toast.LENGTH_SHORT).show();

                        startActivity(new
Intent(RegisterActivity.this, LoginActivity.class));
                        finish();
                    })
                    .addOnFailureListener(e ->
Toast.makeText(this, "Ошибка сохранения данных: " + e.getMessage(),
Toast.LENGTH_LONG).show());
            } else {
                Toast.makeText(this, "Ошибка регистрации: " +
task.getException().getMessage(), Toast.LENGTH_LONG).show();
            }
        });
    }

```

```

    }
    });
}

private void loadGroupsFromFirestore() {
db.collection("groups").get().addOnSuccessListener(queryDocumentSnapshots ->
{
    groupList.clear();
    if (queryDocumentSnapshots.isEmpty()) {
        // Добавляем стандартные группы, если их нет
        addDefaultGroups();
    } else {
        for (DocumentSnapshot doc : queryDocumentSnapshots) {
            String groupName = doc.getString("name");
            if (groupName != null) groupList.add(groupName);
        }
        groupAdapter.notifyDataSetChanged();
        if (groupList.isEmpty()) {
            Toast.makeText(this, "Нет доступных групп. Обратитесь к
администратору.", Toast.LENGTH_LONG).show();
            btnRegister.setEnabled(false);
        } else {
            btnRegister.setEnabled(true);
        }
    }
}).addOnFailureListener(e ->
    Toast.makeText(this, "Ошибка загрузки групп: " + e.getMessage(),
Toast.LENGTH_LONG).show()
);
}

private void addDefaultGroups() {
    Map<String, Object> groupA = new HashMap<>();
    groupA.put("name", "11А");
    Map<String, Object> groupB = new HashMap<>();
    groupB.put("name", "11Б");
    Map<String, Object> groupV = new HashMap<>();
    groupV.put("name", "11В");
    db.collection("groups").document("group1").set(groupA);
    db.collection("groups").document("group2").set(groupB);
    db.collection("groups").document("group3").set(groupV);
    .addOnSuccessListener(aVoid -> loadGroupsFromFirestore());
}
}

```

Листинг 6 RegisterActivity

```

package com.example.electronicgradebook;

public class Student {
    private String id;
    private String lastName;
    private String firstName;
    private String groupId;

    public Student(String id, String lastName, String firstName, String
groupId) {
        this.id = id;
        this.lastName = lastName;
        this.firstName = firstName;
        this.groupId = groupId;
    }
}

```

```

    public String getId() {
        return id;
    }

    public String getLastName() {
        return lastName;
    }

    public String getFirstName() {
        return firstName;
    }

    public String getGroupId() {
        return groupId;
    }

    public String getFullName() {
        return lastName + " " + firstName;
    }

    @Override
    public String toString() {
        return getFullName();
    }
}

```

Листинг 7 Student

```

package com.example.electronicgradebook;

import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.HorizontalScrollView;
import android.widget.LinearLayout;
import android.widget.RadioButton;
import android.widget.RadioGroup;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
import com.example.electronicgradebook.R;
import com.example.electronicgradebook.Group;
import com.example.electronicgradebook.Student;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Date;
import java.util.List;
import java.util.Locale;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.QueryDocumentSnapshot;

public class StudentActivity extends AppCompatActivity {
    private static final String TAG = "StudentActivity";
    private LinearLayout subjectsContainer;
    private LinearLayout datesContainer;
    private TextView textClassInfo;
    private TextView textMonthYear;
    private String studentId;
}

```

```

private String groupId;
private String selectedDate;
private List<Button> dateButtons = new ArrayList<>();
private String selectedSubject;

private static final SimpleDateFormat DATE_FORMAT = new
SimpleDateFormat("yyyy-MM-dd", Locale.getDefault());
private static final SimpleDateFormat MONTH_YEAR_FORMAT = new
SimpleDateFormat("LLLL yyyy", new Locale("ru"));
private static final SimpleDateFormat DAY_FORMAT = new
SimpleDateFormat("d", Locale.getDefault());

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_student);

    Log.d(TAG, "onCreate started");

    // Получаем ID студента из Intent
    studentId = getIntent().getStringExtra("STUDENT_ID");
    Log.d(TAG, "Student ID from intent: " + studentId);

    if (studentId == null) {
        Log.e(TAG, "No student ID provided");
        Toast.makeText(this, "Ошибка: не указан ID студента",
Toast.LENGTH_SHORT).show();
        StudentActivity.this.finish();
        return;
    }

    // Инициализация views
    subjectsContainer = findViewById(R.id.subjectsContainer);
    datesContainer = findViewById(R.id.datesContainer);
    textClassInfo = findViewById(R.id.textClassInfo);
    textMonthYear = findViewById(R.id.textMonthYear);

    if (subjectsContainer == null || datesContainer == null ||
textClassInfo == null || textMonthYear == null) {
        Log.e(TAG, "Could not find required views");
        return;
    }

    // Загружаем профиль студента из Firestore
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection("users").document(studentId).get()
        .addOnSuccessListener(documentSnapshot -> {
            if (!documentSnapshot.exists()) {
                Log.e(TAG, "Профиль студента не найден");
                Toast.makeText(this, "Профиль студента не найден",
Toast.LENGTH_SHORT).show();
                StudentActivity.this.finish();
                return;
            }
            String group = documentSnapshot.getString("group");
            if (group == null || group.isEmpty()) {
                Log.e(TAG, "Группа не указана для студента");
                Toast.makeText(this, "Группа не указана для студента",
Toast.LENGTH_SHORT).show();
                StudentActivity.this.finish();
                return;
            }
            groupId = group;
            Log.d(TAG, "Loaded group for student: " + groupId);

```

```

        // Устанавливаем информацию о классе
        textClassInfo.setText(groupId);
        textMonthYear.setText(MONTH_YEAR_FORMAT.format(new Date()));

        // Генерируем кнопки с датами
        generateDateButtons();
    })
    .addOnFailureListener(e -> {
        Log.e(TAG, "Ошибка загрузки профиля: " + e.getMessage());
        Toast.makeText(this, "Ошибка загрузки профиля: " +
e.getMessage(), Toast.LENGTH_SHORT).show();
        StudentActivity.this.finish();
    });
}

private void generateDateButtons() {
    datesContainer.removeAllViews();
    dateButtons.clear();

    // Создаем кнопки для каждого дня сентября 2024
    Calendar calendar = Calendar.getInstance();
    calendar.set(2024, Calendar.SEPTEMBER, 1);

    // Создаем контейнер для горизонтальной прокрутки
    HorizontalScrollView scrollView = new HorizontalScrollView(this);
    LinearLayout.LayoutParams scrollParams = new
LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT,
        LinearLayout.LayoutParams.WRAP_CONTENT
    );
    scrollView.setLayoutParams(scrollParams);

    // Создаем контейнер для кнопок
    LinearLayout buttonsContainer = new LinearLayout(this);
    buttonsContainer.setOrientation(LinearLayout.HORIZONTAL);
    buttonsContainer.setLayoutParams(new LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.WRAP_CONTENT,
        LinearLayout.LayoutParams.WRAP_CONTENT
    ));

    // Добавляем кнопки для каждого дня
    while (calendar.get(Calendar.MONTH) == Calendar.SEPTEMBER) {
        // Пропускаем выходные
        int dayOfWeek = calendar.get(Calendar.DAY_OF_WEEK);
        if (dayOfWeek != Calendar.SATURDAY && dayOfWeek !=
Calendar.SUNDAY) {
            Button dateButton = new Button(this);
            dateButton.setText(DAY_FORMAT.format(calendar.getTime()));

            // Сохраняем дату в миллисекундах в теге кнопки
            dateButton.setTag(calendar.getTimeInMillis());

            // Устанавливаем обработчик нажатия
            dateButton.setOnClickListener(v -> {
                // Получаем дату из тега кнопки
                long dateInMillis = (long) v.getTag();
                Calendar cal = Calendar.getInstance();
                cal.setTimeInMillis(dateInMillis);
                selectedDate = DATE_FORMAT.format(cal.getTime());

                // Обновляем UI
                for (Button btn : dateButtons) {

```

```

btn.setBackgroundTintList (ContextCompat.getColorStateList (this,
R.color.purple_500));
    }

dateButton.setBackgroundTintList (ContextCompat.getColorStateList (this,
R.color.purple_700));

        // Загружаем расписание
        loadSchedule (selectedDate);
    });

    // Добавляем кнопку в контейнер
    LinearLayout.LayoutParams buttonParams = new
LinearLayout.LayoutParams (
        LinearLayout.LayoutParams.WRAP_CONTENT,
        LinearLayout.LayoutParams.WRAP_CONTENT
    );
    buttonParams.setMargins (8, 0, 8, 0);
    dateButton.setLayoutParams (buttonParams);
    buttonsContainer.addView (dateButton);
    dateButtons.add (dateButton);
}

// Переходим к следующему дню
calendar.add (Calendar.DAY_OF_MONTH, 1);
}

// Добавляем контейнер с кнопками в ScrollView
scrollView.addView (buttonsContainer);
datesContainer.addView (scrollView);

// Выбираем первую кнопку по умолчанию
if (!dateButtons.isEmpty()) {
    dateButtons.get (0).performClick();
}
}

private void loadSchedule (String date) {
    Log.d (TAG, "Loading schedule for date: " + date + ", group: " +
groupId);
    subjectsContainer.removeAllViews();
    selectedSubject = null;

    try {
        Calendar calendar = Calendar.getInstance();
        calendar.setTime (DATE_FORMAT.parse (date));

        String[] daysOfWeek = {"Воскресенье", "Понедельник", "Вторник",
"Среда", "Четверг", "Пятница", "Суббота"};
        String dayOfWeek = daysOfWeek [calendar.get (Calendar.DAY_OF_WEEK)
- 1];

        Log.d (TAG, "Day of week: " + dayOfWeek);

        List<String> subjects =
MockData.getSubjectsForGroupAndDay (groupId, dayOfWeek);
        if (!subjects.isEmpty()) {
            showSubjects (subjects, date);
        } else {
            // Если в MockData нет предметов — пробуем загрузить из
Firestore

            FirebaseFirestore db = FirebaseFirestore.getInstance();
            db.collection ("schedule")
                .whereEqualTo ("groupId", groupId)
                .whereEqualTo ("dayOfWeek", dayOfWeek)

```

```

        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            List<String> firebaseSubjects = new ArrayList<>();
            for (QueryDocumentSnapshot doc :
queryDocumentSnapshots) {
                String subject = doc.getString("subject");
                if (subject != null && !subject.isEmpty()) {
                    firebaseSubjects.add(subject);
                }
            }
            showSubjects(firebaseSubjects, date);
        })
        .addOnFailureListener(e -> {
            Toast.makeText(this, "Ошибка загрузки расписания из
Firebase", Toast.LENGTH_SHORT).show();
            showSubjects(new ArrayList<>(), date);
        });
    }
} catch (Exception e) {
    Toast.makeText(this, "Ошибка при загрузке расписания",
Toast.LENGTH_SHORT).show();
}

private void showSubjects(List<String> subjects, String date) {
    if (subjects.isEmpty()) {
        TextView noSubjectsText = new TextView(this);
        noSubjectsText.setText("В этот день нет занятий");
        noSubjectsText.setTextSize(16);
        noSubjectsText.setPadding(16, 16, 16, 16);
        subjectsContainer.addView(noSubjectsText);
        return;
    }
    for (String subject : subjects) {
        View subjectView =
getLayoutInflater().inflate(R.layout.item_subject_schedule,
subjectsContainer, false);
        TextView subjectNameText =
subjectView.findViewById(R.id.textSubjectName);
        if (subjectNameText == null) continue;
        subjectNameText.setText(subject);
        subjectView.setOnClickListener(v -> {
            selectedSubject = subject;
            showStudentAttendance(subject, date);
        });
        subjectsContainer.addView(subjectView);
    }
}

private void showStudentAttendance(String subject, String date) {
    subjectsContainer.removeAllViews();
    TextView subjectHeader = new TextView(this);
    subjectHeader.setText(subject);
    subjectHeader.setTextSize(20);
    subjectHeader.setTextColor(ContextCompat.getColor(this,
R.color.black));
    subjectHeader.setPadding(16, 16, 16, 24);
    subjectsContainer.addView(subjectHeader);

    // Получаем список студентов группы
    List<Student> students = MockData.getStudentsByGroup(groupId);
    Log.d(TAG, "Showing attendance for " + students.size() + " students
in group " + groupId);
}

```



```

        for (Student student : students) {
            View studentView =
getLayoutInflater().inflate(R.layout.item_student_attendance,
subjectsContainer, false);
            TextView studentNameText =
studentView.findViewById(R.id.textStudentName);
            RadioGroup attendanceGroup =
studentView.findViewById(R.id.radioGroupAttendance);

            if (studentNameText == null || attendanceGroup == null) {
                Log.e(TAG, "Could not find views in student attendance
layout");
                continue;
            }

            studentNameText.setText(student.getFullName());

            // Получаем сохраненную отметку
            String savedMark = MockData.getAttendance(groupId, date,
subject, student.getId());
            Log.d(TAG, "Student " + student.getFullName() + " attendance: "
+ savedMark);

            // Устанавливаем сохраненную отметку, если она есть
            if (savedMark != null) {
                switch (savedMark) {
                    case "present":
                        attendanceGroup.check(R.id.radioPresent);
                        break;
                    case "absent":
                        attendanceGroup.check(R.id.radioAbsent);
                        break;
                    case "excused":
                        attendanceGroup.check(R.id.radioExcused);
                        break;
                }
            }

            // Отключаем возможность изменения отметок для студентов
            RadioButton radioPresent =
studentView.findViewById(R.id.radioPresent);
            RadioButton radioAbsent =
studentView.findViewById(R.id.radioAbsent);
            RadioButton radioExcused =
studentView.findViewById(R.id.radioExcused);
            if (radioPresent != null) radioPresent.setEnabled(false);
            if (radioAbsent != null) radioAbsent.setEnabled(false);
            if (radioExcused != null) radioExcused.setEnabled(false);

            subjectsContainer.addView(studentView);
            Log.d(TAG, "Added student view for: " + student.getFullName());
        }

        Button backButton = new Button(this);
        backButton.setText("Назад к расписанию");
        backButton.setOnClickListener(v -> loadSchedule(date));
        subjectsContainer.addView(backButton);
    }
}

```

Листинг 8 StudentActivity

```

package com.example.electronicgradebook;

```

```

import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class TeacherActivity extends AppCompatActivity {
    private Button btnMarkStudents, btnViewGroups;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_teacher);

        // Initialize buttons
        btnMarkStudents = findViewById(R.id.btnMarkStudents);
        btnViewGroups = findViewById(R.id.btnViewGroups);

        // Set click listeners
        btnMarkStudents.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(TeacherActivity.this,
MarkStudentsActivity.class);
                startActivity(intent);
            }
        });

        btnViewGroups.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(TeacherActivity.this,
ViewGroupsActivity.class);
                startActivity(intent);
            }
        });
    }
}

```

Листинг 9 TeacherActivity

```

package com.example.electronicgradebook;

public class User {
    private String id;
    private String name;
    private String email;
    private String role;
    private String group;

    public User() {}

    public User(String id, String name, String email, String role, String
group) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.role = role;
        this.group = group;
    }

    public String getId() { return id; }
}

```

```

    public void setId(String id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }

    public String getGroup() { return group; }
    public void setGroup(String group) { this.group = group; }
}

```

Листинг 10 User

```

package com.example.electronicgradebook;

import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.ImageButton;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

public class ViewGroupsActivity extends AppCompatActivity {
    private static final String TAG = "ViewGroupsActivity";
    private RecyclerView recyclerView;
    private GroupAdapter adapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_view_groups);

        recyclerView = findViewById(R.id.recyclerViewGroups);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));

        // Загружаем список групп
        loadGroups();
    }

    private void loadGroups() {
        Map<String, Group> groupsMap = MockData.getGroups();
        List<Group> groups = new ArrayList<>(groupsMap.values());
        Log.d(TAG, "Loaded " + groups.size() + " groups");

        // Добавляем логирование для каждой группы
        for (Group group : groups) {
            List<Student> students =

```

```

MockData.getStudentsByGroup(group.getId());
        Log.d(TAG, "Group " + group.getName() + " has " +
students.size() + " students");
        for (Student student : students) {
            Log.d(TAG, "Student in group: " + student.getFullName() + "
(ID: " + student.getId() + ")");
        }
    }

    adapter = new GroupAdapter(groups);
    recyclerView.setAdapter(adapter);
}

private class GroupAdapter extends
RecyclerView.Adapter<GroupAdapter.GroupViewHolder> {
    private List<Group> groups;

    public GroupAdapter(List<Group> groups) {
        this.groups = groups;
    }

    @Override
    public GroupViewHolder onCreateViewHolder(ViewGroup parent, int
viewType) {
        View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_group, parent, false);
        return new GroupViewHolder(view);
    }

    @Override
    public void onBindViewHolder(GroupViewHolder holder, int position) {
        Group group = groups.get(position);
        holder.bind(group);
    }

    @Override
    public int getItemCount() {
        return groups.size();
    }

    class GroupViewHolder extends RecyclerView.ViewHolder {
        private TextView textGroupName;
        private TextView textStudentCount;
        private View studentsContainer;
        private boolean isExpanded = false;

        public GroupViewHolder(View itemView) {
            super(itemView);
            textGroupName = itemView.findViewById(R.id.textGroupName);
            textStudentCount =
itemView.findViewById(R.id.textStudentCount);
            studentsContainer =
itemView.findViewById(R.id.studentsContainer);

            // Обработчик нажатия на карточку группы
            itemView.setOnClickListener(v -> {
                isExpanded = !isExpanded;
                studentsContainer.setVisibility(isExpanded ?
View.VISIBLE : View.GONE);
            });
        }

        public void bind(Group group) {
            // Создаем контейнер для имени группы и кнопки удаления

```

```

        LinearLayout headerRow = new
LinearLayout(itemView.getContext());
        headerRow.setOrientation(LinearLayout.HORIZONTAL);
        headerRow.setLayoutParams(new LinearLayout.LayoutParams(
            LinearLayout.LayoutParams.MATCH_PARENT,
            LinearLayout.LayoutParams.WRAP_CONTENT
        ));
        headerRow.setPadding(16, 16, 16, 8);

        // Создаем новый TextView для названия группы
        TextView groupNameView = new
TextView(itemView.getContext());
        groupNameView.setText(group.getName());
        groupNameView.setTextSize(18);
        groupNameView.setTypeface(null,
android.graphics.Typeface.BOLD);

groupNameView.setTextColor(ContextCompat.getColor(itemView.getContext(),
android.R.color.black));
        groupNameView.setLayoutParams(new LinearLayout.LayoutParams(
            0,
            LinearLayout.LayoutParams.WRAP_CONTENT,
            1.0f
        ));
        headerRow.addView(groupNameView);

        // Кнопка удаления группы
        ImageButton deleteGroupButton = new
ImageButton(itemView.getContext());

deleteGroupButton.setImageResource(android.R.drawable.ic_menu_delete);

deleteGroupButton.setBackgroundResource(android.R.color.transparent);
        deleteGroupButton.setLayoutParams(new
LinearLayout.LayoutParams(
            LinearLayout.LayoutParams.WRAP_CONTENT,
            LinearLayout.LayoutParams.WRAP_CONTENT
        ));
        deleteGroupButton.setOnClickListener(v -> {
            new AlertDialog.Builder(itemView.getContext())
                .setTitle("Удаление группы")
                .setMessage("Вы уверены, что хотите удалить группу "
+ group.getName() + "?\nВсе студенты группы также будут удалены.")
                .setPositiveButton("Да", (dialog, which) -> {
                    MockData.deleteGroup(group.getId());
                    // Обновляем список групп
                    loadGroups();
                })
                .setNegativeButton("Нет", null)
                .show();
        });
        headerRow.addView(deleteGroupButton);

        // Заменяем старый textGroupName на новый headerRow
        ViewGroup parent = (ViewGroup) textGroupName.getParent();
        if (parent != null) {
            int index = parent.indexOfChild(textGroupName);
            parent.removeView(textGroupName);
            parent.addView(headerRow, index);
        }

        // Получаем список студентов группы
        List<Student> students =
MockData.getStudentsByGroup(group.getId());

```

```

        textStudentCount.setText(students.size() + " студентов");

        // Очищаем контейнер студентов
        if (studentsContainer instanceof ViewGroup) {
            ((ViewGroup) studentsContainer).removeAllViews();
        }

        // Добавляем студентов в контейнер
        for (Student student : students) {
            LinearLayout studentRow = new
LinearLayout(itemView.getContext());
            studentRow.setOrientation(LinearLayout.HORIZONTAL);
            studentRow.setLayoutParams(new
LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ));
            studentRow.setPadding(16, 8, 16, 8);

            // Имя студента
            TextView studentView = new
TextView(itemView.getContext());
            studentView.setText(student.getFullName());
            studentView.setTextSize(14);
            studentView.setLayoutParams(new
LinearLayout.LayoutParams(
                0,
                LinearLayout.LayoutParams.WRAP_CONTENT,
                1.0f
            ));

            // Кнопка удаления студента
            ImageButton deleteButton = new
ImageButton(itemView.getContext());

            deleteButton.setImageResource(android.R.drawable.ic_menu_delete);

            deleteButton.setBackgroundResource(android.R.color.transparent);
            deleteButton.setLayoutParams(new
LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.WRAP_CONTENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ));
            deleteButton.setOnClickListener(v -> {
                new AlertDialog.Builder(itemView.getContext())
                    .setTitle("Удаление студента")
                    .setMessage("Вы уверены, что хотите удалить " +
student.getFullName() + "?")
                    .setPositiveButton("Да", (dialog, which) -> {
                        MockData.deleteStudent(student.getId());
                        // Обновляем список студентов
                        bind(group);
                    })
                    .setNegativeButton("Нет", null)
                    .show();
            });

            studentRow.addView(studentView);
            studentRow.addView(deleteButton);
            ((ViewGroup) studentsContainer).addView(studentRow);
        }

        // Скрываем список студентов при переиспользовании
ViewHolder

```

```

        studentsContainer.setVisibility(View.GONE);
        isExpanded = false;
    }
}
}
}

```

Листинг 11 ViewGroupsActivity

Layout

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerViewGroups"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="8dp"
        android:clipToPadding="false" />

    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fabAddGroup"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|end"
        android:layout_margin="16dp"
        android:contentDescription="Добавить группу"
        app:srcCompat="@android:drawable/ic_input_add" />

    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fabAddStudent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|start"
        android:layout_margin="16dp"
        android:contentDescription="Добавить студента"
        app:srcCompat="@android:drawable/ic_menu_add" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

Листинг 12 activity_admin

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <EditText
        android:id="@+id/editEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"

```

```

        android:hint="Email"
        android:inputType="textEmailAddress"
        android:layout_marginBottom="16dp" />

<EditText
    android:id="@+id/editPassword"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Пароль"
    android:inputType="textPassword"
    android:layout_marginBottom="16dp" />

<Button
    android:id="@+id/btnLogin"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Войти"
    android:layout_marginBottom="8dp" />

<Button
    android:id="@+id/btnRegister"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Регистрация" />

<ProgressBar
    android:id="@+id/progressBar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:visibility="gone"
    android:layout_marginTop="24dp" />
</LinearLayout>

```

Листинг 13 activity_login

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- Информация о классе и месяце -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_marginBottom="8dp"
        android:gravity="center_vertical">

        <TextView
            android:id="@+id/textClassInfo"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="18sp"
            android:textStyle="bold"
            android:layout_marginEnd="16dp" />

        <TextView
            android:id="@+id/textMonthYear"
            android:layout_width="wrap_content"

```



```
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:textStyle="bold" />

</LinearLayout>

<!-- Кнопки дат -->
<HorizontalScrollView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    android:scrollbars="none"
    android:overScrollMode="never">

    <LinearLayout
        android:id="@+id/datesContainer"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:paddingStart="4dp"
        android:paddingEnd="4dp" />

</HorizontalScrollView>

<!-- Выбор группы и предмета -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginBottom="16dp">

    <Spinner
        android:id="@+id/spinnerGroup"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginEnd="8dp" />

    <Spinner
        android:id="@+id/spinnerSubject"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginStart="8dp" />

</LinearLayout>

<!-- Список студентов -->
<ScrollView
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1">

    <LinearLayout
        android:id="@+id/studentListContainer"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical" />

</ScrollView>

<Button
    android:id="@+id/buttonSave"
    android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Сохранить" />
```

```
</LinearLayout>
```

Листинг 14 activity_mark_students

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp"
        android:gravity="center">

        <EditText
            android:id="@+id/editEmail"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Email"
            android:inputType="textEmailAddress"
            android:layout_marginBottom="16dp" />

        <EditText
            android:id="@+id/editPassword"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Пароль"
            android:inputType="textPassword"
            android:layout_marginBottom="16dp" />

        <EditText
            android:id="@+id/editLastName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Фамилия"
            android:inputType="textPersonName"
            android:layout_marginBottom="16dp" />

        <EditText
            android:id="@+id/editFirstName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Имя"
            android:inputType="textPersonName"
            android:layout_marginBottom="16dp" />

        <TextView
            android:id="@+id/groupLabel"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Группа"
            android:textSize="16sp"
            android:layout_marginBottom="4dp" />

        <Spinner
            android:id="@+id/spinnerGroup"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
```

```

        android:layout_marginBottom="16dp" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Роль"
            android:textSize="16sp"
            android:layout_marginBottom="4dp" />

        <Spinner
            android:id="@+id/spinnerRole"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="24dp" />

        <Button
            android:id="@+id/btnRegister"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Зарегистрироваться" />

        <ProgressBar
            android:id="@+id/progressBar"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:visibility="gone"
            android:layout_marginTop="24dp" />

    </LinearLayout>
</ScrollView>

```

Листинг 15 activity_register

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- Информация о классе и месяце -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_marginBottom="8dp"
        android:gravity="center_vertical">

        <TextView
            android:id="@+id/textClassInfo"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="18sp"
            android:textStyle="bold"
            android:layout_marginEnd="16dp" />

        <TextView
            android:id="@+id/textMonthYear"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="18sp"
            android:textStyle="bold" />

    </LinearLayout>

```

```

<!-- Кнопки дат -->
<HorizontalScrollView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    android:scrollbars="none"
    android:overScrollMode="never">

    <LinearLayout
        android:id="@+id/datesContainer"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:paddingStart="4dp"
        android:paddingEnd="4dp">

        <!-- Кнопки дат будут добавляться программно -->

    </LinearLayout>

</HorizontalScrollView>

<!-- Список предметов -->
<ScrollView
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1">

    <LinearLayout
        android:id="@+id/subjectsContainer"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical" />

</ScrollView>
</LinearLayout>

```

Листинг 16 activity_student

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".TeacherActivity">

    <Button
        android:id="@+id/btnMarkStudents"
        android:layout_width="200dp"
        android:layout_height="wrap_content"
        android:text="Отметить учеников"
        android:textSize="16sp"
        android:padding="12dp"
        app:layout_constraintBottom_toTopOf="@+id/btnViewGroups"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_chainStyle="packed" />

    <Button

```

```

        android:id="@+id/btnViewGroups"
        android:layout_width="200dp"
        android:layout_height="wrap_content"
        android:text="Просмотр групп"
        android:textSize="16sp"
        android:padding="12dp"
        android:layout_marginTop="16dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/btnMarkStudents" />
    </androidx.constraintlayout.widget.ConstraintLayout>

```

Листинг 17 activity_teacher

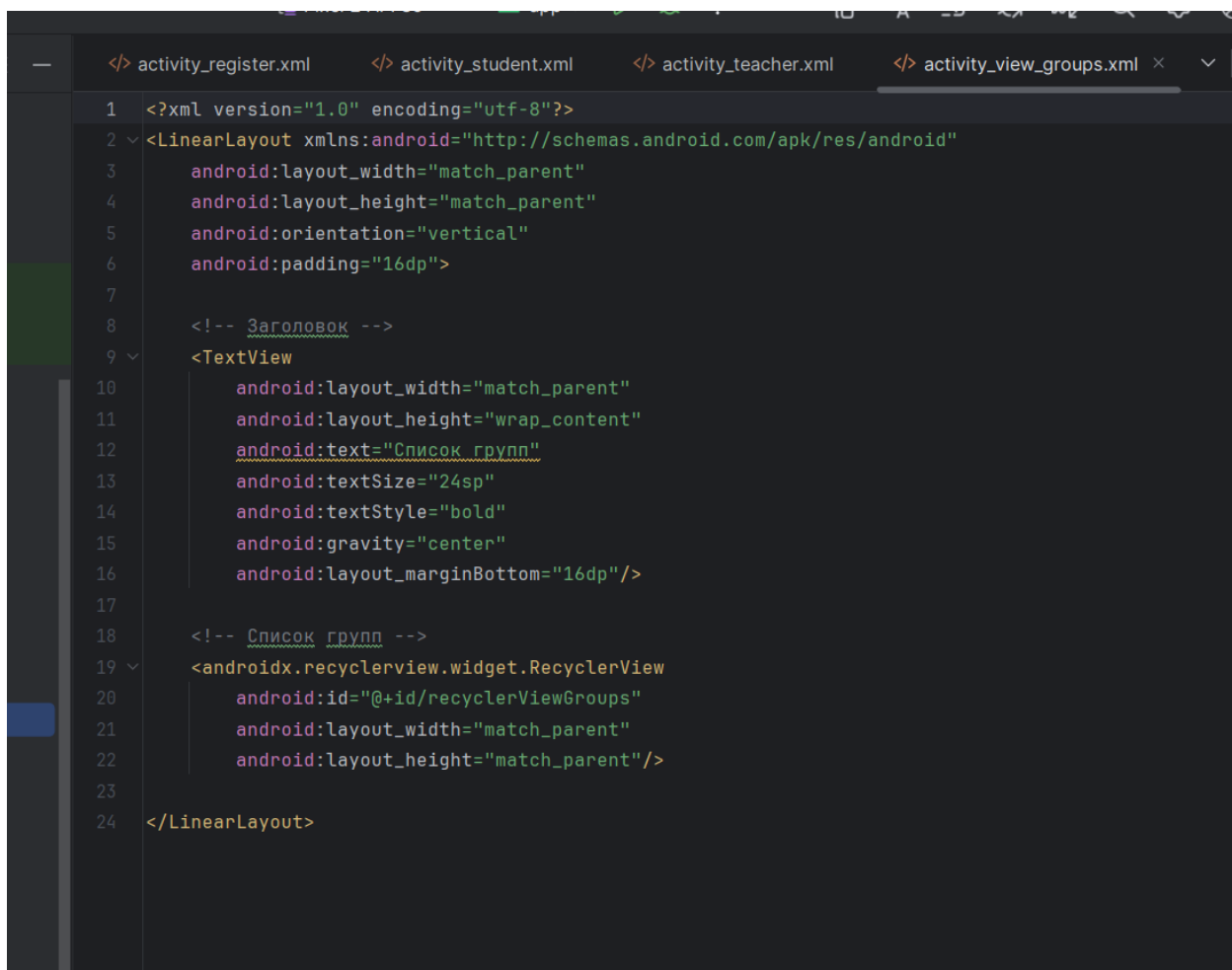


Рисунок 10 dialog_add_groups

```
dialog_add_group.xml    </> activity_student.xml    </> activity_teacher.xml    </> activity_view_groups.xml    </> dialog_add_group.xml
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="wrap_content"
5      android:orientation="vertical"
6      android:padding="16dp">
7
8      <com.google.android.material.textfield.TextInputLayout
9          android:layout_width="match_parent"
10         android:layout_height="wrap_content"
11         android:hint="Название группы">
12
13         <com.google.android.material.textfield.TextInputEditText
14             android:id="@+id/editGroupName"
15             android:layout_width="match_parent"
16             android:layout_height="wrap_content"
17             android:inputType="text"
18             android:maxLength="1" />
19
20     </com.google.android.material.textfield.TextInputLayout>
21
22 </LinearLayout>
```

Рисунок 11 dialog_add_group

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"
        android:hint="Фамилия">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/editLastName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPersonName"
            android:maxLength="1" />

    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"
        android:hint="Имя">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/editFirstName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPersonName"
            android:maxLines="1" />

    </com.google.android.material.textfield.TextInputLayout>

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:layout_marginBottom="8dp"
        android:text="Выберите группу:"
        android:textSize="16sp" />

    <Spinner
        android:id="@+id/spinnerGroup"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp" />

</LinearLayout>

```

Листинг 18 dialog_add_student

```

<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="8dp"
    app:cardElevation="4dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <!-- Название группы -->
        <TextView
            android:id="@+id/textGroupName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textSize="18sp"
            android:textStyle="bold"
            android:textColor="@color/black"/>

        <!-- Количество студентов -->
        <TextView
            android:id="@+id/textStudentCount"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:textColor="@android:color/darker_gray"/>
    
```

```

        <!-- Список студентов (изначально скрыт) -->
        <LinearLayout
            android:id="@+id/studentsContainer"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:layout_marginTop="8dp"
            android:visibility="gone"/>

    </LinearLayout>

</com.google.android.material.card.MaterialCardView>

```

Листинг 19 item_group

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="12dp"
    android:gravity="center_vertical">

    <TextView
        android:id="@+id/textStudentName"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="16sp"
        android:textColor="@color/black" />

    <RadioGroup
        android:id="@+id/radioGroupAttendance"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <RadioButton
            android:id="@+id/radioAbsent"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="✗" />

        <RadioButton
            android:id="@+id/radioPresent"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="✓" />

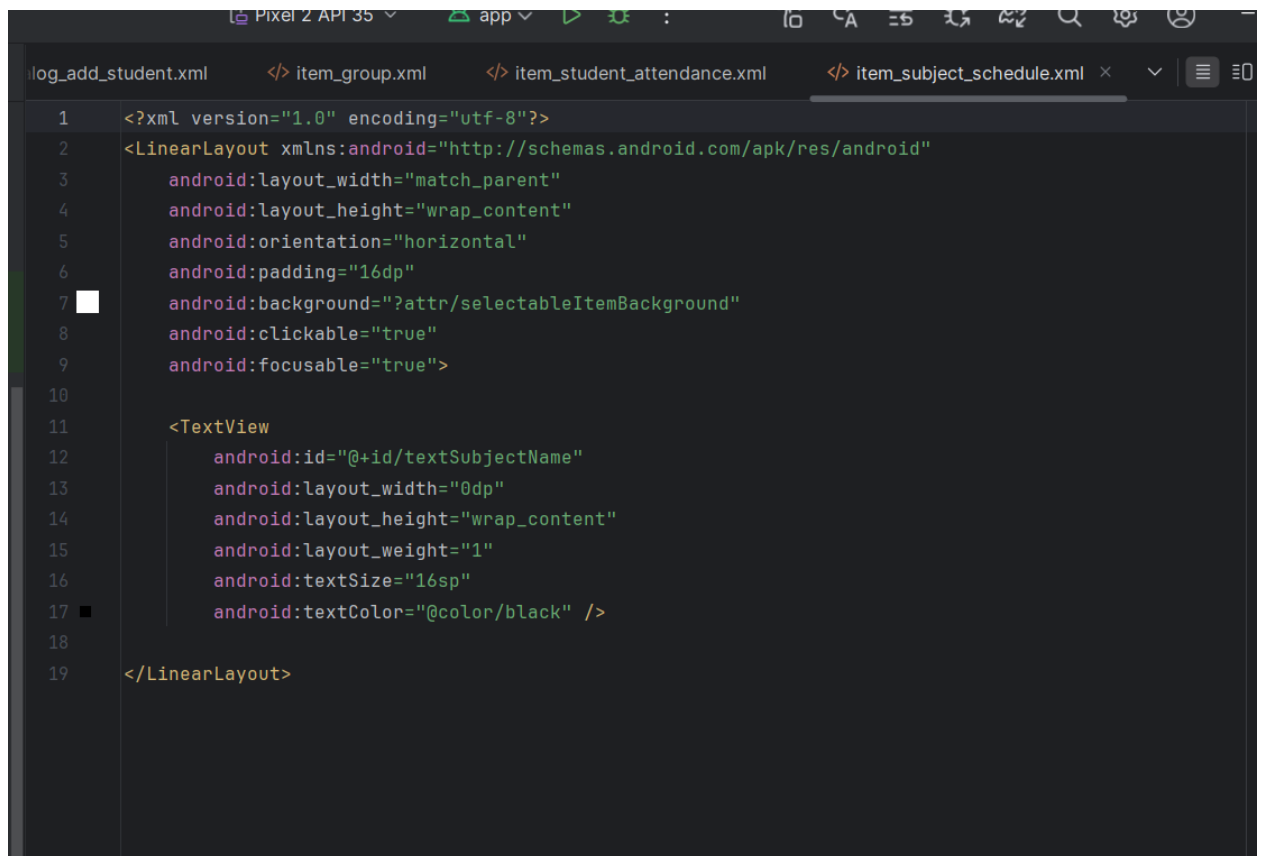
        <RadioButton
            android:id="@+id/radioExcused"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="⌘" />

    </RadioGroup>

</LinearLayout>

```

Листинг 20 item_student_attendance



```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="wrap_content"
5      android:orientation="horizontal"
6      android:padding="16dp"
7      android:background="?attr/selectableItemBackground"
8      android:clickable="true"
9      android:focusable="true">
10
11      <TextView
12          android:id="@+id/textSubjectName"
13          android:layout_width="0dp"
14          android:layout_height="wrap_content"
15          android:layout_weight="1"
16          android:textSize="16sp"
17          android:textColor="@color/black" />
18
19  </LinearLayout>
```

Рисунок 12 item_subject_schedule